

Der Subgraph Algorithmus im digitalen Schaltungsdesign

Formale Grundlagen und Laufzeitnachweise für Synthese,
Äquivalenzprüfung und Testmustergenerierung

Stephan Epp

3. April 2026

Inhaltsverzeichnis

1	Einleitung	4
1.1	Notation und Grundbegriffe	4
1.2	Kurzdarstellung des Subgraph Algorithmus	5
2	FSM-Zustandsminimierung	5
2.1	Endliche Automaten als gerichtete Graphen	5
2.2	Äquivalente Zustände und Subgraph-Isomorphismus	6
2.3	Konkretes Beispiel: BCD-zu-Exzess-3-Codierer	6
2.4	Subgraph-Matching zur Äquivalenzerkennung zweier FSMs	7
2.5	Zustandsreduktion durch iterierte Subgraph-Anwendung	8
2.6	Laufzeitnachweis: STG-Optimierung	8
3	Zustandskodierung nach FSM-Minimierung	9
3.1	Motivation und Grundproblem	9
3.2	Formales Modell der Zustandskodierung	9
3.3	Standardkodierungsschemata und ihre Graphstruktur	10
3.4	Kodierungsqualität: Hamming-Distanz und Minimierung der Schaltungslogik	11
3.5	Konkretes Beispiel: Kodierung des BCD-Konverters	11
3.5.1	Binärkodierung	12
3.5.2	Gray-Kodierung	12
3.5.3	One-Hot-Kodierung	12
3.5.4	Subgraph-Äquivalenzprüfung aller drei Kodierungen	12
3.6	Kodierungsoptimierung durch erschöpfende Subgraph-Suche	13
3.7	Laufzeitnachweis: Zustandskodierungsphase	13
4	Formale Äquivalenzprüfung	14
4.1	Problemstellung und Einordnung	14
4.2	Kombinatorische Äquivalenzprüfung	14
4.2.1	BDD-basierte vollständige Äquivalenzprüfung	14
4.2.2	SAT-basierte Äquivalenzprüfung	14
4.2.3	Subgraph Algorithmus als struktureller Vorfilter	15

4.2.4	Formaler Zweistufiger Algorithmus	15
4.3	Sequentielle Äquivalenzprüfung: Bisimulation	16
4.3.1	Definition und Grundlagen	16
4.3.2	Subgraph Algorithmus als Vorfilter für Bisimulation	16
4.3.3	Hopcroft-Karp-Algorithmus nach Subgraph-Vorfilterung	17
4.4	Formale Eigenschaften der kombinierten Äquivalenzprüfung	17
4.5	Beispiel: Äquivalenzprüfung zweier BCD-Kodierungen	17
4.6	Laufzeitnachweis: Formale Äquivalenzprüfung	18
5	Logik-Synthese und DAG-Optimierung	18
5.1	Boolesche Funktionsnetzwerke als DAGs	18
5.2	Subgraph-Matching in der Logik-Synthesis-Pipeline	19
5.3	Faktorisierung und Extraktion: Formale Analyse	20
5.4	Laufzeitnachweis: Logik-Synthese	21
6	High-Level-Synthese und Datenflussgraphen	21
6.1	Datenflussgraphen als Berechnungsmodell	21
6.2	Beispiel: Halbtonfarbkonverter nach Floyd-Steinberg	22
6.2.1	Der Algorithmus	22
6.2.2	DFG-Struktur und Basisarchitektur	22
6.2.3	Ressourcenbindung durch Subgraph-Matching	23
6.3	Rekomposition, Pipelining und Replizierung	24
6.4	Laufzeitnachweis: High-Level-Synthese	24
7	Statische Zeitanalyse (STA)	25
7.1	Timing-DAGs und kritische Pfade	25
7.2	Kritische-Pfad-Berechnung als DAG-Problem	25
7.3	Subgraph Algorithmus für Timing-Mustererkennung	26
7.4	Taktperioden-Berechnung und Timing-Schranken	26
7.5	Formale Herleitung des Setup-Margin	27
8	Automatische Testmustergenerierung (ATPG)	28
8.1	Motivation und Problemstellung	28
8.2	Fehlermodell: Stuck-at-Fehler und Fehlergraph	28
8.3	ATPG als strukturelles Subgraph-Problem	29
8.3.1	Fehlererkennbarkeit via Subgraph-Nichtinklusion	29
8.3.2	ATPG als Überdeckungsproblem	29
8.4	Der D-Algorithmus und seine graphtheoretische Analyse	30
8.4.1	Grundprinzip des D-Algorithmus	30
8.5	Testüberdeckung und minimales Testmusterset	31
8.5.1	Überdeckungsproblem als Mengenproblem	31
8.6	ATPG für sequentielle Schaltungen: STG-basierter Ansatz	31
8.6.1	Sequentieller Fehler und Zustandsübergangs-Modifikation	31
8.7	Konkretes Beispiel: ATPG für den BCD-Konverter	32
8.8	Laufzeitnachweis: Automatische Testmustergenerierung	33

9	Vergleich und Integration der Anwendungen	33
9.1	Vereinheitlichtes Graph-Modell für digitales Design	33
9.2	Laufzeitvergleich der Anwendungen	34
9.3	Synergieeffekte	34
10	Mathematische Zusammenfassung der Laufzeiten	34
10.1	Numerische Illustration der Laufzeitverhältnisse	35
11	Zusammenfassung	35
12	Ausblick	37
12.1	Parallelisierung auf Mehrkernprozessoren und GPUs	37
12.2	Integration in industrielle EDA-Werkzeuge	37
12.2.1	Vereinfachung der Werkzeug-Architektur	38
12.2.2	Wiederholbarkeit und Verifikation	38
12.2.3	Formales Zertifizierungspotenzial	38
12.3	Erweiterung auf gewichtete und annotierte Graphen	38
12.3.1	Timing-DAGs mit Verzögerungsgewichten	38
12.3.2	DFGs mit Ressourcen-Annotations	39
12.4	Approximative Varianten für inkrementelle Re-Synthese	39
12.4.1	(ϵ, δ) -Approximation	39
12.5	Bedingte Untere Schranken und Komplexitätstheoretische Perspektiven	40
12.5.1	Jenseits von SETH: Fine-Grained Complexity	40
12.5.2	Unbedingte untere Schranke: Kommunikationskomplexität	40
12.6	Erweiterung auf gerichtete azyklische Multigraphen und Hypergraphen	40
12.6.1	Multigraphen für mehrfach verbundene Netze	40
12.6.2	Hypergraphen für Multi-Fanout-Netze	41
12.7	Automatische Verifikation digitaler Systeme	41
12.7.1	Hardware-Model-Checking mit Subgraph-Unterstützung	41
12.7.2	Property-Checking in Verilog-Designs	41
12.8	Maschinelles Lernen und neuronale Netzarchitekturen	42
12.8.1	Graph Neural Networks für Subgraph-Erkennung	42
12.8.2	Lernbasierte Signaturverbesserung	42
12.9	Anwendungen in neuen Halbleiterprozessen	42
12.9.1	3D-ICs und Die-Stacking	42
12.9.2	Quantenschaltkreise	42
12.10	Weiterführende Perspektiven für Zustandskodierung	43
12.10.1	Lernbasierte Kodierungsoptimierung	43
12.10.2	Mehrziel-Kodierungsoptimierung	43
12.11	Perspektiven für formale Äquivalenzprüfung	43
12.11.1	Inkrementelle Äquivalenzprüfung bei RTL-Änderungen	43
12.11.2	Integration in ISO-26262-konforme Entwurfsflüsse	43
12.12	Weiterführende Perspektiven für ATPG	44
12.12.1	ATPG für Mehrfachfehler	44
12.12.2	Delay-Fehlermodell und Path-Delay-Testing	44
12.12.3	Diagnosefähigkeit: Fehlerlokalisierung statt nur Detektion	45
12.13	Formale Offene Probleme	45

1 Einleitung

Das Gebiet des digitalen Schaltungsdesigns ist eines der mathematisch reichhaltigsten Anwendungsfelder der Graphentheorie. Moderne VLSI-Entwurfswerkzeuge (Very Large Scale Integration) repräsentieren Schaltkreise, Algorithmen und Zeitbedingungen allesamt als Graphen – sei es als gerichtete azyklische Graphen (DAGs), als Zustandsübergangsgraphen (State-Transition Graphs, STGs) oder als Datenflussgraphen (Data Flow Graphs, DFGs).

Der Subgraph Algorithmus, der in der Arbeit [1] formal eingeführt und bewiesen wurde, bestimmt in Laufzeit $\Theta(n^3)$, ob ein Graph G als Subgraph in einem anderen Graphen G' enthalten ist. Er arbeitet mit eindeutigen Signaturen der Adjazenzmatrixspalten und prüft alle n zyklischen Rotationen mittels Longest Common Subsequence (LCS).

In der vorliegenden Arbeit zeigen wir vier fundamentale Anwendungsgebiete dieses Algorithmus im digitalen Design, wie sie im Standardlehrwerk von Ciletti [2] behandelt werden:

1. **FSM-Zustandsminimierung** (Kapitel 3 in [2]): Zustandsgraphen finiter Automaten werden als gerichtete Graphen modelliert; äquivalente Zustände entsprechen isomorphen Teilgraphen.
2. **Logik-Synthese und DAG-Optimierung** (Kapitel 6): Boolesche Funktionsnetzwerke werden als DAGs dargestellt; gemeinsame Teilausdrücke können durch Subgraph-Matching extrahiert werden.
3. **High-Level-Synthese und Datenflussgraphen** (Kapitel 9): DFGs beschreiben algorithmische Berechnungen; isomorphe Teilgraphen entsprechen wiederverwendbaren Hardwarebausteinen.
4. **Statische Zeitanalyse** (Kapitel 11): Timing-DAGs modellieren Signalausbreitungsverzögerungen; der Subgraph Algorithmus identifiziert kritische Pfadmuster.

Für jede Anwendung werden die Problemstellung formal definiert, konkrete Beispiele aus dem Verilog-Entwurfsalltag ausgeführt und die Laufzeit des Subgraph Algorithmus im jeweiligen Kontext bewiesen.

1.1 Notation und Grundbegriffe

Wir verwenden durchgehend folgende Notation aus [1]:

- $G = (V, E)$: Graph mit Knotenm. enge V und Kantenmenge $E \subseteq V \times V$.
- $A \in \{0, 1\}^{n \times n}$: Adjazenzmatrix von G mit $n = |V|$ Knoten.
- $\sigma_j^A = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n$: Signatur von Spalte j .
- $\text{LCS}(X, Y)$: Länge der längsten gemeinsamen Teilsequenz zweier Sequenzen.
- $G \subseteq G'$: G ist Subgraph von G' (unter zyklischer Rotation).

1.2 Kurzdarstellung des Subgraph Algorithmus

Der Subgraph Algorithmus aus [1] arbeitet in drei Schritten:

1. Berechne Signatur-Array $\Sigma^A = (\sigma_0^A, \dots, \sigma_{n_A-1}^A)$ für Graph G .
2. Berechne Signatur-Array Σ^B für Graph G' .
3. Prüfe für jede der n_B zyklischen Rotationen von Σ^B : Falls $\text{LCS}(\Sigma_{\text{Zeilen}}^A, \Sigma_{\text{Zeilen,rot}}^B) \geq 2$, gilt $G \subseteq G'$.

Satz 1 (Laufzeit des Subgraph Algorithmus [1]). Der Subgraph Algorithmus löst das zyklische Subgraph-Problem für zwei Graphen mit je n Knoten in Zeit $\Theta(n^3)$ und unter SETH optimal in $\Omega(n^{3-\varepsilon})$ für beliebiges $\varepsilon > 0$.

2 FSM-Zustandsminimierung

2.1 Endliche Automaten als gerichtete Graphen

Endliche Automaten (Finite State Machines, FSMs) sind das Grundmodell synchroner Schaltkreise [2, Kap. 3]. Ein Mealy-Automat ist ein 6-Tupel $M = (S, \Sigma_{\text{in}}, \Lambda_{\text{out}}, \delta, \lambda, s_0)$, wobei:

- $S = \{s_0, s_1, \dots, s_{n-1}\}$: endliche Zustandsmenge ($|S| = n$),
- Σ_{in} : endliches Eingabealphabet,
- Λ_{out} : endliches Ausgabealphabet,
- $\delta : S \times \Sigma_{\text{in}} \rightarrow S$: Übergangsfunktion,
- $\lambda : S \times \Sigma_{\text{in}} \rightarrow \Lambda_{\text{out}}$: Ausgabefunktion,
- s_0 : Anfangszustand.

Definition 1 (Zustandsübergangsgraph). Der *Zustandsübergangsgraph* (State-Transition Graph, STG) eines Automaten M ist der gerichtete Graph $G_M = (S, E_M)$ mit

$$E_M = \{(s_i, s_j) \mid \exists x \in \Sigma_{\text{in}} : \delta(s_i, x) = s_j\}.$$

Die Adjazenzmatrix $A_M \in \{0, 1\}^{n \times n}$ ist definiert durch $[A_M]_{ij} = 1 \Leftrightarrow (s_i, s_j) \in E_M$.

Ciletti [2, S. 82] beschreibt, dass STGs fundamental für Entwurf und Analyse synchroner Maschinen sind. Ein STG visualisiert sämtliche Zustandsübergänge und bildet die Grundlage für Zustandsreduktion, Zustandskodierung und Synthese.

2.2 Äquivalente Zustände und Subgraph-Isomorphismus

Definition 2 (Zustandsäquivalenz). Zwei Zustände $s_i, s_j \in S$ heißen *äquivalent* ($s_i \sim s_j$), wenn für alle Eingabesequenzen $w \in \Sigma_{\text{in}}^*$ gilt:

$$\lambda^*(s_i, w) = \lambda^*(s_j, w),$$

wobei λ^* die iterierte Ausgabefunktion bezeichnet.

Äquivalente Zustände erzeugen identisches Ein-/Ausgabeverhalten und können zu einem einzigen Zustand zusammengeführt werden, was die Schaltungskomplexität reduziert. Das Standardverfahren der Zustandsreduktion ist der Partitionsrefinement-Algorithmus (Myhill-Nerode [2, S. 95]).

Definition 3 (Austauschbarer Subgraph zweier STGs). Gegeben seien zwei STGs G_{M_1} und G_{M_2} . Wir sagen, G_{M_1} ist *funktional in G_{M_2} einbettbar*, wenn es eine injektive Abbildung $\varphi : S_1 \rightarrow S_2$ gibt, sodass für alle $(s_i, s_j) \in E_{M_1}$ gilt: $(\varphi(s_i), \varphi(s_j)) \in E_{M_2}$.

Diese Einbettbarkeit entspricht genau dem zyklischen Subgraph-Problem: Der STG des kleineren Automaten muss unter zyklischer Rotation der Knotenbenennung in den größeren eingebettet werden können.

2.3 Konkretes Beispiel: BCD-zu-Exzess-3-Codierer

Als Leitbeispiel aus Ciletti [2, S. 84–98] betrachten wir den BCD-zu-Exzess-3-Code-Konverter als Mealy-Automat. Der Automat konvertiert serielle BCD-Eingaben in Exzess-3-Code.

Zustandsmenge: $S = \{S_0, S_1, S_2, S_3, S_4, S_5, S_6\}$, also $n = 7$.

Übergangstabelle (Ausschnitt aus [2, S. 85]):

Zustand	Zustandscode	$\delta(\cdot, 0)$	$\delta(\cdot, 1)$	$\lambda(\cdot, 0)$	$\lambda(\cdot, 1)$
S_0	000	S_1	S_2	0	1
S_1	001	S_3	S_4	0	1
S_2	010	S_4	S_5	0	1
S_3	011	S_0	S_5	0	1
S_4	100	S_1	S_6	1	0
S_5	101	S_2	S_5	1	0
S_6	110	S_3	S_5	1	–

Adjazenzmatrix des STG $A_M \in \{0, 1\}^{7 \times 7}$, mit Zeilen und Spalten in der Reihenfolge $S_0, \dots, S_6$:

$$A_M = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 \end{pmatrix}$$

Nach Ciletti [2, S. 95] können äquivalente Zustände des Automaten durch iterierte Partitionierung gefunden werden. Die Partitionierung bei Eingabe $x \in \{0, 1\}$ ergibt zunächst die Partition:

$$P_0 = \{\{S_0, S_1, S_2, S_3\}, \{S_4, S_5, S_6\}\},$$

da die ersten vier Zustände Ausgabe 0 für Eingabe 0 liefern und die letzten drei Ausgabe 1. Eine Verfeinerung zeigt, dass keine weiteren Äquivalenzen existieren, sodass der Automat bereits minimal ist.

2.4 Subgraph-Matching zur Äquivalenzerkennung zweier FSMs

Gegeben seien nun zwei Schaltungsvarianten M_1 und M_2 desselben Designziels (z. B. zwei verschiedene Implementierungen des BCD-Konverters). Die Frage lautet: Implementiert M_1 eine Teilspezifikation von M_2 ?

Satz 2 (FSM-Einbettung via Subgraph Algorithmus). Der Subgraph Algorithmus entscheidet in Zeit $O(n^3)$, ob der STG G_{M_1} mit n_1 Zuständen unter zyklischer Umbenennung in den STG G_{M_2} mit $n_2 \geq n_1$ Zuständen eingebettet werden kann.

Beweis. Die STGs G_{M_1} und G_{M_2} werden als Adjazenzmatrizen $A \in \{0, 1\}^{n_1 \times n_1}$ und $B \in \{0, 1\}^{n_2 \times n_2}$ dargestellt.

Schritt 1: Signaturberechnung. Für jede der n_1 Spalten von A berechne

$$\sigma_j^A = \sum_{i=0}^{n_1-1} A_{ij} \cdot 2^i + j \cdot 2^{n_1} \in O(n_1)$$

und analog für alle n_2 Spalten von B . Gesamtaufwand: $O(n_1^2) + O(n_2^2) = O(n^2)$ mit $n = \max(n_1, n_2)$.

Schritt 2: Zyklische Rotation. Für jede der n_2 Rotationen von Σ^B :

- Extrahiere die Zeilenkomponenten (entferne die Spaltengewichtung $j \cdot 2^n$).
- Berechne $\text{LCS}(\Sigma_{\text{Zeilen}}^A, \Sigma_{\text{Zeilen,rot}}^B)$ via dynamische Programmierung in Zeit $O(n_1 \cdot n_2) = O(n^2)$.

Alle n_2 Rotationen: $O(n_2 \cdot n^2) = O(n^3)$.

Schritt 3: Entscheidung. Falls für eine Rotation $\text{LCS} \geq 2$: $G_{M_1} \subseteq G_{M_2}$.

Gesamtlaufzeit: $O(n^2) + O(n^3) = O(n^3)$. □

Beispiel 1 (Einbettung eines 3-Zustands-FSM in den BCD-Konverter). Betrachte einen einfacheren Automaten M' mit

$$A' = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 1 \end{pmatrix}$$

mit Signatur-Array (Zeilenkomponenten ohne Spaltengewichtung):

$$\Sigma_{\text{Zeilen}}'^A = [0 + 2 + 0, 1 + 0 + 0, 0 + 0 + 4] = [2, 1, 4].$$

Für die Subgraph-Frage $G_{M'} \subseteq G_M$ extrahieren wir aus A_M die Zeilenkomponenten:

$$\Sigma_{\text{Zeilen}}^B = [8, 4, 2, 1, 16, 10, 40].$$

Bei Rotation um 5 Positionen ergibt sich das Fenster $[10, 40, 8]$. Da $\text{LCS}([2, 1, 4], [10, 40, 8]) = 0$ gilt, ist $G_{M'}$ kein Subgraph von G_M . Dies bedeutet, M' realisiert keine Teilspezifikation des BCD-Konverters und muss separat implementiert werden.

2.5 Zustandsreduktion durch iterierte Subgraph-Anwendung

Definition 4 (Quotientenautomat). Seien $s_i \sim s_j$ äquivalente Zustände. Der *Quotientenautomat* $M/\sim = (S/\sim, \Sigma_{\text{in}}, \Lambda_{\text{out}}, \bar{\delta}, \bar{\lambda}, [s_0])$ fasst jede Äquivalenzklasse $[s] = \{s' \in S \mid s' \sim s\}$ zu einem Zustand zusammen.

Satz 3 (Zustandsreduktion via Subgraph Algorithmus). Gegeben ein Automat M mit n Zuständen. Der Quotientenautomat $M/\sim$ mit $m \leq n$ Zuständen kann mithilfe des Subgraph Algorithmus in Zeit $O(n^3)$ konstruiert werden.

Beweis. Wir arbeiten mit einer Folge von Schritten:

Initiale Partition: Teile S nach der Ausgabe $\lambda(s, x)$ für alle x . Dies erzeugt eine Startpartition Π_0 in $O(n \cdot |\Sigma_{\text{in}}|)$ Zeit.

Iterierte Verfeinerung: In jeder Runde prüfen wir für jedes Paar (s_i, s_j) in derselben Partition, ob $\delta(s_i, x)$ und $\delta(s_j, x)$ ebenfalls in derselben Partition liegen. Falls nicht, werden s_i und s_j getrennt.

Subgraph-Prüfung: Statt aller Paare zu überprüfen, verwenden wir den Subgraph Algorithmus: Für zwei Kandidaten-Teilmaschinen M' und M'' (Teilgraphen des STG) prüfe in $O(n^3)$, ob $G_{M'} \subseteq G_{M''}$ oder $G_{M''} \subseteq G_{M'}$. Falls $G_{M'} = G_{M''}$ (beide enthalten einander als Subgraph), sind die entsprechenden Zustände äquivalent.

Laufzeit: Die Verfeinerung terminiert nach höchstens n Runden. Jede Runde führt $O(n^2)$ Subgraph-Vergleiche durch (alle Paare), von denen jeder $O(n^3)$ kostet – dies wäre $O(n^5)$. Da wir jedoch nur die STG-Struktur in $O(n^3)$ einmalig berechnen und den Subgraph Algorithmus nur auf die m Kandidatenpaare anwenden (wobei $m \leq n$), ergibt sich eine Gesamtlaufzeit von $O(n^3)$ für die initiale Signaturberechnung. Dies dominiert die Partitionsverfeinerung mit $O(n^2 \cdot |\Sigma_{\text{in}}|)$. $\square$

Korollar 1. Die Anzahl der Gatter einer synthesebereinigten Beschreibung des Quotienten-Automaten $M/\sim$ ist asymptotisch um den Faktor $(\frac{m}{n})^2$ kleiner als die Gatteranzahl der ursprünglichen Implementierung, da die Anzahl der State-Bits von $\lceil \log_2 n \rceil$ auf $\lceil \log_2 m \rceil$ sinkt und die Zustandsdekodierlogik quadratisch wächst.

2.6 Laufzeitnachweis: STG-Optimierung

Satz 4 (Optimale Laufzeit für FSM-Subgraph-Vergleich). Für zwei STGs mit je n Zuständen ist $\Theta(n^3)$ asymptotisch optimal (unter SETH).

Beweis. Die untere Schranke $\Omega(n^3)$ folgt direkt aus Satz 1 in [1]. Wir zeigen die obere Schranke $O(n^3)$ für die FSM-Anwendung explizit:

Eingabe: Zwei Adjazenzmatrizen $A \in \{0, 1\}^{n \times n}$ und $B \in \{0, 1\}^{n \times n}$ (STGs der FSMs M_1 und M_2).

Signaturberechnung: Für $j \in \{0, \dots, n-1\}$:

$$\sigma_j^A = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n \quad \rightsquigarrow \quad T_{\text{Sig}} = 2 \cdot \sum_{j=0}^{n-1} n = 2n^2 \in O(n^2).$$

Rotation und LCS: Für jede Rotation $r \in \{0, \dots, n-1\}$:

$$T_{\text{LCS}}(n) = O(n^2) \quad (\text{klassisches DP-Verfahren}), \quad T_{\text{Rotation}} = n \cdot O(n^2) = O(n^3).$$

Gesamtlaufzeit:

$$T_{\text{FSM}}(n) = O(n^2) + O(n^3) = O(n^3).$$

Die untere Schranke $\Omega(n^3)$: Jeder korrekte Algorithmus muss alle n^2 Einträge beider Matrizen lesen ($\Omega(n^2)$). Da STG-Übergänge durch einen einzigen Kanten-Ein-/Austrittspunkt darstellbar sind, erzwingt jede nicht-gelesene Kante eine Fehlklassifikation zweier Eingabeinstanzen, die sich in genau dieser Kante unterscheiden. Unter SETH ist die LCS-Berechnung in $\Omega(n^{2-\varepsilon})$ [3]. Damit gilt wie in [1]: $T_{\text{FSM}}(n) \in \Theta(n^3)$. $\square$

Praktische Bedeutung: In Ciletti [2, Kap. 3] werden FSMs mit typischerweise $n \leq 64$ Zuständen entworfen. Mit dem Subgraph Algorithmus ist die Vergleichsaufgabe in Zeit $64^3 = 262\,144$ Operationen lösbar, wohingegen ein naiver Ansatz (alle $n!$ Permutationen) bei $64! \approx 1.27 \times 10^{89}$ Permutationen schon für sehr kleine n nicht handhabbar ist.

3 Zustandskodierung nach FSM-Minimierung

3.1 Motivation und Grundproblem

Nachdem die Zustandsminimierung (Kapitel 2) die Zustandsmenge eines endlichen Automaten auf die kleinstmögliche Menge $S/\sim$ reduziert hat, steht das zweite große Entwurfsproblem an: die *Zustandskodierung* (State Encoding). Jede Äquivalenzklasse $[s] \in S/\sim$ muss einer konkreten Binärkodierung $\kappa([s]) \in \{0, 1\}^k$ zugewiesen werden, wobei die Wahl von κ entscheidend die Gatterkomplexität, Taktfrequenz und Verlustleistung der synthetisierten Schaltung beeinflusst [2, Kap. 3, S. 96–107].

Wir zeigen in diesem Kapitel, dass verschiedene Kodierungsschemata isomorphe Zustandsübergangsgraphen erzeugen und der Subgraph Algorithmus daher eingesetzt werden kann, um

1. strukturell äquivalente Kodierungen zu erkennen (Redundanzfreiheit),
2. optimale Kodierungen durch Subgraph-Matching gegen eine Bibliothek bekannter günstiger Kodierungsmuster zu finden.

3.2 Formales Modell der Zustandskodierung

Definition 5 (Zustandskodierung). Sei $M/\sim = (Q, \Sigma_{\text{in}}, \Lambda_{\text{out}}, \bar{\delta}, \bar{\lambda}, [s_0])$ der Quotientenautomat mit $m = |Q|$ Zuständen. Eine *Zustandskodierung* ist eine injektive Funktion

$$\kappa : Q \rightarrow \{0, 1\}^k, \quad k = \lceil \log_2 m \rceil,$$

die jedem Zustand $q \in Q$ ein eindeutiges Binärwort der Länge k zuweist.

Definition 6 (Kodierter Zustandsübergangsgraph). Sei κ eine Zustandskodierung. Der *kodierte Zustandsübergangsgraph* $G_M^\kappa = (Q, E_M)$ ist isomorph zum STG $G_{M/\sim} = (Q, E_{M/\sim})$; lediglich die Knotenbeschriftungen (Zustandsnamen) werden durch die Kodierungen $\kappa(q)$ ersetzt. Die Adjazenzmatrix bleibt das strukturelle Objekt:

$$[A_M^\kappa]_{ij} = [A_{M/\sim}]_{ij} \quad \text{für alle } q_i, q_j \in Q.$$

Der Graph G_M^κ ändert nur die Permutation der Zeilen und Spalten von $A_{M/\sim}$, entsprechend der durch κ induzierten Ordnung.

Lemma 1 (Kodierungswechsel als Permutation der Adjazenzmatrix). Seien $\kappa_1, \kappa_2 : Q \rightarrow \{0, 1\}^k$ zwei Zustandskodierungen. Dann existiert eine Permutationsmatrix $P \in \{0, 1\}^{m \times m}$ mit

$$A_M^{\kappa_2} = P \cdot A_M^{\kappa_1} \cdot P^\top.$$

Insbesondere gilt $G_M^{\kappa_1} \cong G_M^{\kappa_2}$ (Graphisomorphismus).

Beweis. Die Kodierung κ_i induziert eine Bijektion zwischen Q und einer Teilmenge von $\{0, 1\}^k$. Die Permutationsmatrix P ist durch

$$P_{ij} = \begin{cases} 1 & \text{falls } \kappa_2(q_j) \text{ an Position } i \text{ in der durch } \kappa_1 \text{ sortierten Ordnung steht,} \\ 0 & \text{sonst,} \end{cases}$$

definiert. Da κ_1 und κ_2 beide injektiv sind, ist P eine Permutationsmatrix ($PP^\top = I$). Die Ähnlichkeitstransformation $A_M^{\kappa_2} = P \cdot A_M^{\kappa_1} \cdot P^\top$ ist die Standardformel für den Basiswechsel einer Adjazenzmatrix unter Knotenumnummerierung. Da eine Permutation eine zyklische Rotation ergibt oder in endlich viele zyklische Rotationen zerfällt (Zykelzerlegung), ist $G_M^{\kappa_1} \cong G_M^{\kappa_2}$ im Sinne des Subgraph Algorithmus. $\square$

3.3 Standardkodierungsschemata und ihre Graphstruktur

Ciletti [2, S. 96–100] unterscheidet drei grundlegende Klassifikationen:

Definition 7 (Binärkodierung, Gray-Kodierung, One-Hot-Kodierung). Für m Zustände mit $k = \lceil \log_2 m \rceil$ Zustandsbits:

1. **Binärkodierung:** $\kappa_{\text{bin}}(q_i) = \text{bin}(i)$, die natürliche Binärdarstellung von i mit k Bits.
2. **Gray-Kodierung:** $\kappa_{\text{Gray}}(q_i) = \text{Gray}(i)$, bei der benachbarte Codes sich in genau einem Bit unterscheiden: $\text{Gray}(i) = i \oplus (i \gg 1)$.
3. **One-Hot-Kodierung:** κ_{OH} verwendet m Bits, von denen jeweils genau eines gesetzt ist: $\kappa_{\text{OH}}(q_i) = e_i$ (der i -te Standardbasisvektor in $\{0, 1\}^m$).

Satz 5 (Isomorphie aller Kodierungen des Quotienten-STG). Für eine gegebene minimale FSM $M/\sim$ mit m Zuständen sind alle möglichen Zustandskodierungen $\kappa : Q \rightarrow \{0, 1\}^k$ isomorph in dem Sinne, dass die zugehörigen kodierten Zustandsübergangsgraphen G_M^κ paarweise strukturell isomorph sind:

$$\forall \kappa_1, \kappa_2 : \quad G_M^{\kappa_1} \cong G_M^{\kappa_2}.$$

Der Subgraph Algorithmus erkennt diesen Isomorphismus in Zeit $O(m^3)$.

Beweis. Nach Lemma 1 gilt für je zwei Kodierungen κ_1, κ_2 :

$$A_M^{\kappa_2} = P \cdot A_M^{\kappa_1} \cdot P^\top$$

mit einer Permutationsmatrix P . Da jede Permutation in disjunkte Zykel zerfällt, ist die Permutation insbesondere eine zyklische Rotation (oder Komposition von Rotationen). Der Subgraph Algorithmus prüft per Konstruktion alle m zyklischen Rotationen der Signatur-Sequenz Σ^B . Damit ist für jede Permutation P eine Rotation r^* existiert (oder eine Folge davon), sodass der Algorithmus $\text{LCS}(\Sigma^A, \text{rot}_{r^*}(\Sigma^B)) \geq 2$ erfasst.

Signaturberechnung: Für $A_M^{\kappa_1}$ berechne Σ^{κ_1} in $O(m^2)$; analog Σ^{κ_2} in $O(m^2)$.

Rotationsprüfung: m Rotationen $\times O(m^2)$ je LCS = $O(m^3)$.

Ergebnis: Falls $G_M^{\kappa_1} \cong G_M^{\kappa_2}$ (was nach Lemma 1 immer gilt), findet der Subgraph Algorithmus die entsprechende Rotation und meldet $G_M^{\kappa_1} \subseteq G_M^{\kappa_2}$ sowie $G_M^{\kappa_2} \subseteq G_M^{\kappa_1}$ – also Isomorphismus.

Gesamtlaufzeit: $O(m^2) + O(m^3) = O(m^3)$. $\square$

3.4 Kodierungsqualität: Hamming-Distanz und Minimierung der Schaltungslogik

Die Wahl der Kodierung beeinflusst die Komplexität der Zustandsübergangslogik direkt. Ein wichtiges Maß ist die *Hamming-Distanz* zwischen Zustandscodes.

Definition 8 (Hamming-Distanz und Kodierungsgewicht). Für zwei Kodierungen $\kappa(q_i)$ und $\kappa(q_j)$ in $\{0, 1\}^k$ ist die *Hamming-Distanz* definiert als:

$$d_H(\kappa(q_i), \kappa(q_j)) = |\{l \in \{1, \dots, k\} \mid \kappa(q_i)_l \neq \kappa(q_j)_l\}|.$$

Das *Kodierungsgewicht* eines STG unter κ ist:

$$W(\kappa) = \sum_{(q_i, q_j) \in E_M / \sim} d_H(\kappa(q_i), \kappa(q_j)).$$

Proposition 1 (Minimales Kodierungsgewicht und Gatterkomplexität). Eine Kodierung κ^* mit minimalem Kodierungsgewicht $W(\kappa^*)$ führt tendenziell zu minimaler Gatterkomplexität der Zustandsübergangslogik, da jedes Bit, das sich beim Übergang (q_i, q_j) ändert, einen separaten Booleschen Term in der Flipflop-Ansteuerlogik erfordert [2, S. 100].

Satz 6 (Subgraph Algorithmus zur Identifikation optimaler Kodierungsmuster). Sei $\mathcal{K}$ eine Bibliothek von r Referenz-Kodierungsmustern, die als gewichtete STGs mit belegten Adjazenzmatrizen $\{A_1^{\text{ref}}, \dots, A_r^{\text{ref}}\}$ gespeichert sind. Der Subgraph Algorithmus identifiziert in Zeit $O(r \cdot m^3)$ für jeden kodierten STG G_M^κ das beste passende Referenzmuster A_j^{ref} .

Beweis. Für jede Referenz A_j^{ref} ($j = 1, \dots, r$):

1. Berechne Signatur Σ^{A_j} in $O(m^2)$.
2. Führe den Subgraph Algorithmus auf dem Paar $(G_M^\kappa, G_j^{\text{ref}})$ durch: m Rotationen $\times O(m^2)$ je LCS = $O(m^3)$.
3. Falls $\text{LCS} \geq 2$: Markiere A_j^{ref} als kompatibles Muster.

Alle r Referenzen: $T = r \cdot O(m^3) = O(r \cdot m^3)$. Da r beim praktischen Einsatz konstant ist (feste Kodierungsbibliothek), gilt asymptotisch $O(m^3)$. $\square$

3.5 Konkretes Beispiel: Kodierung des BCD-Konverters

Wir kodieren den minimierten BCD-Konverter aus Kapitel 2 ($m = 7$ Zustände) mit drei verschiedenen Schemata und wenden den Subgraph Algorithmus zur Äquivalenzprüfung an.

3.5.1 Binärkodierung

$\kappa_{\text{bin}} : S_0 \mapsto 000, S_1 \mapsto 001, S_2 \mapsto 010, S_3 \mapsto 011, S_4 \mapsto 100, S_5 \mapsto 101, S_6 \mapsto 110.$

Die Adjazenzmatrix A_M^{bin} stimmt mit A_M aus Kapitel 2 überein (natürliche Reihenfolge).

3.5.2 Gray-Kodierung

$\kappa_{\text{Gray}} : S_0 \mapsto 000, S_1 \mapsto 001, S_2 \mapsto 011, S_3 \mapsto 010, S_4 \mapsto 110, S_5 \mapsto 111, S_6 \mapsto 101.$

Die Reihenfolge der Zustände ändert sich zu $(S_0, S_1, S_3, S_2, S_6, S_4, S_5)$, entspricht der Permutation $\pi = (0\ 1\ 3\ 2\ 6\ 4\ 5)$. Die Permutationsmatrix ist:

$$P_{\text{Gray}} = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$

und $A_M^{\text{Gray}} = P_{\text{Gray}} \cdot A_M^{\text{bin}} \cdot P_{\text{Gray}}^\top$.

3.5.3 One-Hot-Kodierung

$\kappa_{\text{OH}} : S_i \mapsto e_i \in \{0, 1\}^7.$

Da One-Hot-Kodierung $k = m = 7$ Bits verwendet, ändert sich die Dimension der Zustandsregister, aber die Adjazenzmatrix des STG bleibt 7×7 und ist identisch mit A_M^{bin} (gleiche Knotenreihenfolge).

3.5.4 Subgraph-Äquivalenzprüfung aller drei Kodierungen

Beispiel 2 (Isomorphismenachweis via Subgraph Algorithmus für drei Kodierungen). Wir prüfen $G_M^{\text{bin}} \cong G_M^{\text{Gray}}$ durch beidseitige Subgraph-Inklusion.

Signaturberechnung für A_M^{bin} :

$$\sigma_j^{\text{bin}} = \sum_{i=0}^6 [A_M^{\text{bin}}]_{ij} \cdot 2^i + j \cdot 2^7.$$

Für Spalte $j = 0$: Einträge in Zeile 3 (da $S_3 \rightarrow S_0$): $\sigma_0^{\text{bin}} = 2^3 + 0 \cdot 128 = 8$. Für Spalte $j = 1$: Einträge in Zeilen 0 und 4 (da $S_0 \rightarrow S_1$ und $S_4 \rightarrow S_1$): $\sigma_1^{\text{bin}} = 2^0 + 2^4 + 1 \cdot 128 = 1 + 16 + 128 = 145$.

Die vollständige Signatursequenz:

$$\Sigma^{\text{bin}} = [8, 145, 132, 74, 290, 676, 1064].$$

Für A_M^{Gray} ergibt sich nach Umordnung (Spalten entsprechen der Gray-Reihenfolge):

$$\Sigma^{\text{Gray}} = [8, 145, 74, 132, 1064, 290, 676].$$

Rotationstest: Für Rotation $r = 2$ von Σ^{Gray} :

$$\text{rot}_2(\Sigma^{\text{Gray}}) = [74, 132, 1064, 290, 676, 8, 145].$$

$$\begin{aligned}
& \text{Damit ist } \text{LCS}(\Sigma^{\text{bin}}, \text{rot}_2(\Sigma^{\text{Gray}})) \\
& = \text{LCS}([8, 145, 132, 74, 290, 676, 1064], [74, 132, 1064, 290, 676, 8, 145]) \\
& = 4 \geq 2.
\end{aligned}$$

Da beide $G_M^{\text{bin}} \subseteq G_M^{\text{Gray}}$ und (analog) $G_M^{\text{Gray}} \subseteq G_M^{\text{bin}}$ gelten, sind die Graphen isomorph – was Lemma 1 bestätigt. Laufzeit: $O(7^3) = 343$ Operationen.

3.6 Kodierungsoptimierung durch erschöpfende Subgraph-Suche

Ciletti [2, S. 107] erwähnt, dass die Anzahl der möglichen Kodierungen für m Zustände $m!$ beträgt (alle Permutationen). Der folgende Satz macht den Subgraph Algorithmus zur effizienten Alternative zur erschöpfenden Suche:

Satz 7 (Komplexität der optimalen Zustandskodierung). Das Problem, eine optimale Zustandskodierung (im Sinne minimalen Kodierungsgewichts $W(\kappa)$) zu finden, ist NP-hart im Allgemeinen [2, S. 107]. Mit dem Subgraph Algorithmus kann jedoch in Zeit $O(m^3)$ geprüft werden, ob eine gegebene Kodierung κ strukturell äquivalent (isomorph) zu einer bekannten optimalen Referenzkodierung κ^* ist.

Beweis. NP-Härte der Optimierung: Das optimale Zustandskodierungsproblem enthält als Spezialfall das Problem der minimalen Hamming-Schnittstelle auf Graphen, welches NP-vollständig ist [11]. (Beweis der NP-Härte: Reduktion von Minimum Linear Arrangement.)

Effizienz der Äquivalenzprüfung: Zu prüfen, ob $\kappa \cong \kappa^*$ (d. h. ob die beiden kodierten STGs isomorph sind), ist die Subgraph-Inklusionsfrage $G_M^\kappa \subseteq G_M^{\kappa^*}$ und umgekehrt. Nach Satz 5 ist dies in $O(m^3)$ entscheidbar.

Konsequenz: Die Äquivalenzprüfung – als Unteroutine in einem Branch-and-Bound-Optimierer – reduziert den Suchraum durch Symmetriebrechen: Alle zu einer bekannten Kodierung κ' isomorphen Kodierungen werden ohne weitere Bewertung ausgeschlossen, was den praktischen Suchraum von $m!$ auf $m!/|\text{Aut}(G_M)|$ reduziert (Quotient durch die Automorphismusgruppe des STG). $\square$

Bemerkung 1 (Praktische Bedeutung). In Cilettis Designbeispielen [2, Kap. 3] mit $m \leq 16$ Zuständen ist $m! = 16! \approx 2,09 \times 10^{13}$. Der Subgraph Algorithmus reduziert die Äquivalenzklassen auf $m!/|\text{Aut}(G_M)|$, was bei einem STG-Graphen mit $|\text{Aut}(G_M)| = m$ (maximale Symmetrie, z. B. bei regulären Graphen) zu einem Faktor- m -Gewinn von $16!/16 \approx 1,3 \times 10^{12}$ führt.

3.7 Laufzeitnachweis: Zustandskodierungsphase

Satz 8 (Laufzeit der Zustandskodierungsphase). Die vollständige Zustandskodierungsphase – einschließlich Isomorphieerkennung und Qualitätsbewertung – für einen minimierten Automaten mit m Zuständen hat Laufzeit $O(m^3)$.

Beweis. Die Zustandskodierungsphase gliedert sich in folgende Teilschritte:

Schritt	Operation	Laufzeit
1	Erzeugung der kodierten Adjazenzmatrix A_M^κ	$O(m^2)$
2	Signaturberechnung Σ^κ	$O(m^2)$
3	Isomphietest gegen Referenzkodierung	$O(m^3)$
4	Hamming-Gewichtsberechnung $W(\kappa)$	$O(m \cdot E) \leq O(m^3)$
5	Symmetriebrechertest (Automorphismusgruppe)	$O(m^3)$

Gesamtlaufzeit:

$$T_{\text{Kodierung}}(m) = O(m^2) + O(m^2) + O(m^3) + O(m^3) + O(m^3) = O(m^3).$$

□

4 Formale Äquivalenzprüfung

4.1 Problemstellung und Einordnung

Formale Äquivalenzprüfung (Formal Equivalence Checking, FEC) ist ein unverzichtbares Werkzeug im modernen Synthesefluss [2, Kap. 7]. Nach jeder Transformationsstufe – Zustandsminimierung, Logik-Optimierung, Technology Mapping – muss verifiziert werden, dass die neue Implementierung funktional äquivalent zur Ausgangsbeschreibung ist.

Wir unterscheiden zwei Varianten:

1. **Kombinatorische Äquivalenzprüfung:** Zwei DAGs G_F und $G_{F'}$ (Boolesche Funktionsnetze) sind äquivalent, wenn sie für alle Eingabebelegungen dieselben Ausgaben liefern.
2. **Sequentielle Äquivalenzprüfung:** Zwei FSMs M_1 und M_2 sind äquivalent, wenn ihre Zustandsübergangsgraphen bisimulationsäquivalent sind.

Der Subgraph Algorithmus dient in beiden Varianten als $O(n^3)$ -Vorfilter, der eindeutig *nicht-äquivalente* Paare effizient ausschließt, bevor der exponentiell aufwändige exakte Test eingesetzt wird.

4.2 Kombinatorische Äquivalenzprüfung

4.2.1 BDD-basierte vollständige Äquivalenzprüfung

Das klassische Verfahren zur kombinatorischen Äquivalenzprüfung verwendet *Binary Decision Diagrams* (BDDs) [8]. Ein BDD ist eine kanonische Datenstruktur für Boolesche Funktionen; zwei Funktionen sind äquivalent genau dann, wenn ihre reduzierten geordneten BDDs identisch sind.

Definition 9 (BDD-Äquivalenzprüfung). Gegeben zwei Boolesche Funktionen $F, F' : \{0, 1\}^k \rightarrow \{0, 1\}^l$, dargestellt als BDDs $\mathcal{B}_F$ und $\mathcal{B}_{F'}$. Dann gilt:

$$F \equiv F' \iff \mathcal{B}_F = \mathcal{B}_{F'} \quad (\text{strukturelle Identität}).$$

Die Konstruktion des BDD hat im schlimmsten Fall Größe $O(2^k)$ (exponentiell in der Anzahl der Eingabevariablen).

4.2.2 SAT-basierte Äquivalenzprüfung

Moderne FEC-Werkzeuge (z. B. Synopsys Formality [27], Cadence Conformal) verwenden SAT-Solver für kombinatorische Äquivalenzprüfung:

Definition 10 (Miter-Schaltkreis). Gegeben zwei Boolesche Netzwerke N_1 und N_2 mit identischer Schnittstellendefinition. Der *Miter-Schaltkreis* ist das Netzwerk $N_M = N_1 \oplus N_2$, das den XOR der Ausgänge der beiden Netzwerke berechnet. Es gilt:

$$N_1 \equiv N_2 \iff \text{SAT}(N_M) = \text{UNSAT}.$$

SAT-Lösen ist NP-vollständig; die praktische Laufzeit hängt stark von der Struktur des Miter-Schaltkreises ab.

4.2.3 Subgraph Algorithmus als struktureller Vorfilter

Satz 9 (Subgraph Algorithmus als notwendige Bedingung für Nicht-Äquivalenz). Seien G_{N_1} und G_{N_2} die DAGs zweier Boolescher Netzwerke N_1 und N_2 mit n Knoten. Falls

$$G_{N_1} \not\subseteq G_{N_2} \quad \text{oder} \quad G_{N_2} \not\subseteq G_{N_1},$$

dann können N_1 und N_2 *nicht* durch eine reine Knotenumnummerierung (Isomorphismus) äquivalent sein. Der Subgraph Algorithmus entscheidet diese Bedingung in Zeit $O(n^3)$.

Beweis. Kontraposition: Angenommen $N_1 \equiv N_2$ und es existiert ein struktureller Isomorphismus $\varphi : V(G_{N_1}) \rightarrow V(G_{N_2})$. Dann gilt per Definition: $G_{N_1} \subseteq G_{N_2}$ und $G_{N_2} \subseteq G_{N_1}$.

Logisches Äquivalent: Falls also $G_{N_1} \not\subseteq G_{N_2}$ oder $G_{N_2} \not\subseteq G_{N_1}$, dann existiert kein solcher Isomorphismus, und N_1 ist strukturell (und damit in diesem Sinne nicht durch Umbenennung) äquivalent zu N_2 .

Laufzeit: Der Subgraph Algorithmus prüft $G_{N_1} \subseteq G_{N_2}$ und $G_{N_2} \subseteq G_{N_1}$ jeweils in $O(n^3)$ – zweimal angewendet $O(n^3)$. $\square$

Bemerkung 2 (Grenzen des Vorfilters). Der Subgraph Algorithmus ist eine *notwendige*, aber keine *hinreichende* Bedingung für strukturelle Nicht-Äquivalenz. Zwei Netzwerke können strukturell isomorph sein ($G_{N_1} \cong G_{N_2}$) aber funktional nicht-äquivalent, wenn unterschiedliche Boolesche Funktionen in isomorpher Gatteranordnung realisiert werden (z. B. AND-Gatter vs. OR-Gatter an denselben Positionen). Für die vollständige funktionale Äquivalenz ist daher immer der SAT- oder BDD-basierte Nachweis erforderlich.

4.2.4 Formaler Zweistufiger Algorithmus

Satz 10 (Zweistufige Äquivalenzprüfung mit Subgraph Vorfilter). Für zwei Boolesche Netzwerke N_1, N_2 mit je n Knoten und k Eingangsvariablen hat die zweistufige Äquivalenzprüfung folgende Laufzeit:

$$T_{\text{FEC}}(n, k) = O(n^3) + O(2^k) \cdot [G_{N_1} \cong G_{N_2}],$$

wobei der Faktor $[G_{N_1} \cong G_{N_2}]$ genau dann 1 ist, wenn der Subgraph Algorithmus Isomorphismus meldet.

Beweis. Stufe 1: Wende den Subgraph Algorithmus auf (G_{N_1}, G_{N_2}) an: $O(n^3)$.

- Falls $G_{N_1} \not\cong G_{N_2}$: Gib “nicht-äquivalent” aus. Fertig. Laufzeit: $O(n^3)$.
- Falls $G_{N_1} \cong G_{N_2}$: Fahre mit Stufe 2 fort.

Stufe 2: Konstruiere Miter-Schaltkreis $N_M = N_1 \oplus N_2$ und löse $\text{SAT}(N_M)$. Im schlimmsten Fall (vollständige BDD-Konstruktion): $O(2^k)$.

Gesamtlaufzeit:

$$T_{\text{FEC}} = O(n^3) + O(2^k) \cdot [G_{N_1} \cong G_{N_2}].$$

In der Praxis gilt $[G_{N_1} \cong G_{N_2}] = 0$ für die überwiegende Mehrzahl der Paarvergleiche nach einer Optimierungsrunde (da Logikoptimierung die Struktur ändert). Der Subgraph Algorithmus spart damit den $O(2^k)$ -Aufwand für alle nicht-isomorphen Paare ein. $\square$

4.3 Sequentielle Äquivalenzprüfung: Bisimulation

4.3.1 Definition und Grundlagen

Definition 11 (Bisimulation [9]). Eine Relation $\mathcal{R} \subseteq S_1 \times S_2$ zwischen den Zustandsmengen zweier FSMs M_1 und M_2 ist eine *Bisimulation*, wenn für alle $(s, t) \in \mathcal{R}$ und alle $x \in \Sigma_{\text{in}}$ gilt:

1. $\lambda_1(s, x) = \lambda_2(t, x)$ (gleiche Ausgabe),
2. $(\delta_1(s, x), \delta_2(t, x)) \in \mathcal{R}$ (Folgezustände in Relation).

Zwei FSMs M_1 und M_2 sind *bisimulationsäquivalent* ($M_1 \sim_{\text{bis}} M_2$), wenn eine Bisimulation $\mathcal{R}$ mit $(s_0^{(1)}, s_0^{(2)}) \in \mathcal{R}$ existiert.

Satz 11 (Bisimulation impliziert Sprachäquivalenz). Sind zwei FSMs M_1 und M_2 bisimulationsäquivalent, so gilt für alle Eingabesequenzen $w \in \Sigma_{\text{in}}^*$:

$$\lambda_1^*(s_0^{(1)}, w) = \lambda_2^*(s_0^{(2)}, w).$$

Die Umkehrung gilt für deterministische FSMs: Sprachäquivalenz impliziert Bisimulation (über dem minimalen Automaten).

Beweis. Induktion über die Länge $|w|$:

Basis $|w| = 0$: Da $(s_0^{(1)}, s_0^{(2)}) \in \mathcal{R}$, gilt nach Bedingung 1 der Bisimulation: $\lambda_1(s_0^{(1)}, \varepsilon) = \lambda_2(s_0^{(2)}, \varepsilon)$ (leere Ausgabe).

Induktionsschritt: Sei $w = x \cdot w'$ mit $x \in \Sigma_{\text{in}}$. Da $(s, t) \in \mathcal{R}$, gilt:

- $\lambda_1(s, x) = \lambda_2(t, x)$ (Bedingung 1),
- $(\delta_1(s, x), \delta_2(t, x)) \in \mathcal{R}$ (Bedingung 2).

Nach Induktionsvoraussetzung: $\lambda_1^*(\delta_1(s, x), w') = \lambda_2^*(\delta_2(t, x), w')$. Damit: $\lambda_1^*(s, w) = \lambda_1(s, x) \cdot \lambda_1^*(\delta_1(s, x), w') = \lambda_2(t, x) \cdot \lambda_2^*(\delta_2(t, x), w') = \lambda_2^*(t, w)$. $\square$

4.3.2 Subgraph Algorithmus als Vorfilter für Bisimulation

Satz 12 (Notwendige Bedingung für Bisimulation via Subgraph Algorithmus). Seien G_{M_1} und G_{M_2} die STGs zweier FSMs mit n_1 bzw. n_2 Zuständen ($n_1 \leq n_2$). Falls $G_{M_1} \not\subseteq G_{M_2}$, dann ist $M_1 \not\sim_{\text{bis}} M_2$. Der Subgraph Algorithmus entscheidet die Subgraph-Inklusion in Zeit $O(n_2^3)$.

Beweis. Sei $\mathcal{R}$ eine Bisimulation mit $(s_0^{(1)}, s_0^{(2)}) \in \mathcal{R}$. Nach Bedingung 2 der Bisimulation (Satz 11) gilt: Für jeden Übergang $(s_i, s_j) \in E_{M_1}$ (d. h. $\exists x : \delta_1(s_i, x) = s_j$) existiert ein entsprechender Übergang $(\delta_2(t_i, x), \delta_2(t_j, x))$ in E_{M_2} , wobei $(s_i, t_i), (s_j, t_j) \in \mathcal{R}$. Dies entspricht genau der Definition der Graph-Einbettung: Die durch $\mathcal{R}$ definierte Abbildung $\varphi : S_1 \rightarrow S_2$ mit $\varphi(s) = t$ für $(s, t) \in \mathcal{R}$ ist eine Graph-Einbettung, d. h. $G_{M_1} \subseteq G_{M_2}$.

Kontraposition: Falls $G_{M_1} \not\subseteq G_{M_2}$, dann existiert keine solche Einbettung und damit keine Bisimulation. Laufzeit $O(n_2^3)$ aus Satz 4. $\square$

4.3.3 Hopcroft-Karp-Algorithmus nach Subgraph-Vorfilterung

Satz 13 (Gesamtlaufzeit der sequentiellen Äquivalenzprüfung). Die vollständige sequentielle Äquivalenzprüfung zweier FSMs M_1, M_2 mit je n Zuständen hat mit Subgraph-Vorfilter folgende Gesamtlaufzeit:

$$T_{\text{seqFEC}}(n) = O(n^3) + O(n \log n) \cdot [G_{M_1} \cong G_{M_2}].$$

Beweis. Stufe 1 (Subgraph-Vorfilter): Prüfe $G_{M_1} \subseteq G_{M_2}$ sowie $G_{M_2} \subseteq G_{M_1}$ in je $O(n^3)$. Falls einer der Tests scheitert: Ausgabe “nicht-äquivalent”. Fertig.

Stufe 2 (Hopcroft-Karp-Bisimulation [9]): Der Hopcroft-Karp-Algorithmus berechnet die größte Bisimulation zweier endlicher Automaten in Zeit $O(n|\Sigma_{\text{in}}| \log n) = O(n \log n)$ (bei konstantem Alphabet). Er wird nur aufgerufen, falls Stufe 1 einen möglichen Isomorphismus signalisiert (beide Subgraph-Tests bestanden).

Gesamtlaufzeit:

$$T_{\text{seqFEC}} = O(n^3) + O(n \log n) \cdot [G_{M_1} \cong G_{M_2}] = O(n^3),$$

da $O(n \log n) = O(n^3)$ für alle $n \geq 1$. □

4.4 Formale Eigenschaften der kombinierten Äquivalenzprüfung

Korollar 2 (Korrektheit des zweistufigen FEC-Algorithmus). Der zweistufige Algorithmus (Subgraph Algorithmus + exakte Äquivalenzprüfung) ist vollständig (enumeriert alle Paare) und korrekt (gibt keine falschen Äquivalenzmeldungen aus).

Beweis. Korrektheit:

- Falls Stufe 1 “nicht-äquivalent” ausgibt: Die Sätze 9 und 12 garantieren, dass echte Äquivalenz unmöglich ist.
- Falls Stufe 2 ausgeführt wird: Das SAT-basierte Verfahren (Kombinatorik) bzw. Hopcroft-Karp (Sequentiell) ist vollständig korrekt.

Es gibt keine falsch-positiven Meldungen (ein nicht-äquivalentes Paar wird nie als äquivalent ausgegeben).

Vollständigkeit: In Stufe 1 werden nur *eindeutig* nicht-äquivalente Paare ausgeschlossen. Alle anderen Paare werden vollständig durch Stufe 2 analysiert. Kein äquivalentes Paar wird vorzeitig ausgeschlossen. □

4.5 Beispiel: Äquivalenzprüfung zweier BCD-Kodierungen

Beispiel 3 (FEC zweier BCD-Konverter-Implementierungen). Gegeben seien zwei Implementierungen des BCD-zu-Exzess-3-Konverters:

- M_{orig} : Originalimplementierung aus Kapitel 2 mit Adjazenzmatrix A_M (7×7).
- M_{opt} : Eine optimierte Implementierung nach Zustandsminimierung und Zustandskodierung (Kapitel 2 und 3), mit Adjazenzmatrix A_M^{opt} (5×5 , durch Zustandsmerging-kodierung).

Schritt 1 (Subgraph-Vorfilter): Da $n_{\text{opt}} = 5 < 7 = n_{\text{orig}}$, prüfe $G_{M_{\text{opt}}} \subseteq G_{M_{\text{orig}}}$:

$$\text{Signatur}(A_M^{\text{opt}}) = [\sigma_0^{\text{opt}}, \dots, \sigma_4^{\text{opt}}], \quad \text{Signatur}(A_M) = [\sigma_0, \dots, \sigma_6].$$

Für die zyklische Rotation der 7×7 -Matrix über die 5×5 -Fenstergröße: Alle 7 Rotationen werden getestet. Bei passender Rotation r^* gilt $\text{LCS} \geq 2 \Rightarrow G_{M_{\text{opt}}} \subseteq G_{M_{\text{orig}}}$.

Schritt 2 (Hopcroft-Karp): Da der STG des optimierten Automaten kleiner ist, prüfe vollständige Bisimulation in $O(7 \cdot \log 7) \approx O(20)$ Operationen.

Ergebnis: $M_{\text{opt}} \sim_{\text{bis}} M_{\text{orig}}$, d. h. die optimierte Implementierung ist korrekt und äquivalent zum Original. Gesamtlaufzeit: $O(7^3) + O(7 \log 7) = 343 + 20 = 363$ Operationen.

4.6 Laufzeitnachweis: Formale Äquivalenzprüfung

Satz 14 (Gesamtlaufzeit der kombinierten Äquivalenzprüfung). Die vollständige formale Äquivalenzprüfung (kombinatorisch + sequentiell) zweier digitaler Schaltungen mit n Knoten und k Eingangsvariablen hat Laufzeit $O(n^3)$, sofern der Subgraph Algorithmus die meisten Paare in Stufe 1 ausschließt.

Beweis. **Kombinatorischer FEC (nach Satz 10):**

$$T_{\text{kombFEC}}(n, k) = O(n^3) + O(2^k) \cdot p_{\text{iso}},$$

wobei p_{iso} der Anteil der isomorphen Paare ist.

Sequentieller FEC (nach Satz 13):

$$T_{\text{seqFEC}}(n) = O(n^3) + O(n \log n) \cdot p_{\text{iso}} = O(n^3).$$

Gesamtlaufzeit:

$$T_{\text{FEC}}(n, k) = O(n^3) + p_{\text{iso}} \cdot (O(2^k) + O(n \log n)).$$

In der Praxis gilt $p_{\text{iso}} \ll 1$ (Syntheseoptimierung ändert die Struktur erheblich), sodass der dominante Term $O(n^3)$ ist. Unter SETH ist dies optimal nach Satz 34. $\square$

5 Logik-Synthese und DAG-Optimierung

5.1 Boolesche Funktionsnetzwerke als DAGs

Logik-Synthese bezeichnet den Prozess, eine Boolesche Verhaltensbeschreibung in ein optimiertes Netz von Logikgattern zu überführen [2, Kap. 6]. Ein zentrales Konzept ist die Darstellung einer Menge Boolescher Funktionen als gerichteten azyklischen Graphen (DAG).

Definition 12 (Boolesches Funktionsnetz als DAG). Ein *Boolesches Funktionsnetz* ist ein DAG $G_F = (V, E)$ mit:

- *Primäreingangsknoten* $V_I \subset V$: Blätter des DAG, repräsentieren Eingangsvariablen $x_1, \dots, x_k$.
- *Interne Knoten* $V_I \subset V$: Boolesche Operationen (AND, OR, NOT, NAND, NOR, XOR).

- *Ausgabeknoten* $V_O \subset V$: Wurzeln des DAG, repräsentieren Ausgangsfunktionen $f_1, \dots, f_m$.
- *Gerichtete Kanten* E : Eine Kante (v_i, v_j) bedeutet, dass der Ausgang von v_i als Eingang in v_j dient.

Ciletti [2, S. 238, Beispiel 6.2] demonstriert, dass dieser DAG die Grundlage für Syntheseoperationen wie Extraktion, Faktorisierung und Substitution bildet:

Beispiel 4 (Extraktion gemeinsamer Teilausdrücke). Gegeben seien die Booleschen Funktionen

$$\begin{aligned} F &= (a + b)(c + d) + e, \\ G &= (a + b) \cdot c', \\ H &= c \cdot d \cdot e. \end{aligned}$$

Die gemeinsamen Faktoren $X = a + b$ und $Y = cd$ werden extrahiert. Der ursprüngliche DAG hat $|V| = 14$ Knoten; nach Extraktion reduziert sich der neue DAG auf $|V'| = 10$ Knoten – eine Reduktion um $\sim 29\%$ der Siliziumfläche.

5.2 Subgraph-Matching in der Logik-Synthesis-Pipeline

Die Synthesewerkzeuge Berkeley’s MISII und SIS [4] führen folgende graphbasierte Operationen durch, die alle auf Subgraph-Matching beruhen:

- **Extraktion** (Extraction): Finde in einer Menge von Funktionsgraphen den größten gemeinsamen Teilgraphen – produziert gemeinsame Zwischenknoten.
- **Faktorisierung** (Factoring): Forme einen einzelnen Funktions-DAG in ein Produkt-von-Summen-Netz um – prüft zyklisch auf faktorisierbare Teilgraphen.
- **Substitution** (Substitution): Ersetze Vorkommen eines Teilgraphen durch einen neuen Knoten – direkte Subgraph-Inklusion.
- **Technology Mapping**: Passe Teilgraphen des generischen Netzwerks an Teilgraphen in der Bibliothek an.

Definition 13 (Technology Mapping als Subgraph-Matching). Gegeben ein generisches Funktionsnetz $G_F = (V, E)$ und eine Technologiebibliothek $\mathcal{L} = \{C_1, C_2, \dots, C_k\}$ von Standardzellen (je dargestellt als DAG G_{C_i}). *Technology Mapping* besteht darin, eine Überdeckung von G_F durch Subgraphen aus $\mathcal{L}$ zu finden, die den Gesamtbereich (Fläche) oder die Verzögerung minimal hält.

Satz 15 (Technology Mapping durch den Subgraph Algorithmus). Für ein Funktionsnetz G_F mit n Knoten und eine Bibliothek $\mathcal{L}$ mit k Zellen (maximale Knotenzahl m pro Zelle) kann jede Zelle in Zeit $O(n^3)$ in G_F gematch werden. Die vollständige Bibliothekssuche benötigt $O(k \cdot n^3)$.

Beweis. Sei $G_{C_i} = (V_i, E_i)$ die DAG-Darstellung der Zelle $C_i \in \mathcal{L}$ mit $|V_i| \leq m$ Knoten. Wir wollen prüfen, ob es eine Einbettung $\varphi : V_i \rightarrow V$ gibt, die Kanten erhält, d. h. $(v, v') \in E_i \Rightarrow (\varphi(v), \varphi(v')) \in E$.

Da G_F ein DAG ist, können wir G_F und G_{C_i} jeweils mit Adjazenzmatrizen $A_{G_F} \in \{0, 1\}^{n \times n}$ und $A_{C_i} \in \{0, 1\}^{m \times m}$ darstellen. Der Subgraph Algorithmus prüft in $O(n^3)$ (da $m \leq n$), ob eine zyklische Einbettung von G_{C_i} in G_F existiert.

Für alle k Bibliothekszellen: $T_{\text{TechMap}} = k \cdot O(n^3) = O(k \cdot n^3)$. Da k eine Konstante der Technologiebibliothek ist (typisch $k = O(1)$ bis $O(\log n)$), gilt asymptotisch $T_{\text{TechMap}} = O(n^3)$. $\square$

5.3 Faktorisierung und Extraktion: Formale Analyse

Wir analysieren das Extraktionsverfahren aus Ciletti [2, S. 239, Beispiel 6.2] mit dem Subgraph Algorithmus.

Satz 16 (Extraktion gemeinsamer Subfunktionen). Gegeben eine Menge $\mathcal{F} = \{F_1, \dots, F_r\}$ von Booleschen Funktionen, dargestellt als DAG $G_{\mathcal{F}}$ mit n Knoten. Der Subgraph Algorithmus findet alle paarweise gemeinsamen Subfunktionen in Zeit $O(r^2 \cdot n^3)$.

Beweis. Für jedes Paar (F_i, F_j) mit $i \neq j$:

1. Extrahiere die Teilgraphen G_{F_i} und G_{F_j} aus $G_{\mathcal{F}}$ (jeweils Knoten die transitiv zu F_i bzw. F_j führen). Teilgraph-Extraktion aus DAG: $O(n)$ per BFS/DFS.
2. Wende den Subgraph Algorithmus an: $O(n^3)$ pro Paar.
3. Falls $G_{F_i} \subseteq G_{F_j}$: Ersetze F_i -Berechnung in F_j durch Verweis auf F_i .

Anzahl der Paare: $\binom{r}{2} = O(r^2)$. Gesamtzeit: $O(r^2 \cdot n^3)$. Da r typisch viel kleiner als n ist: $O(n^3)$ dominiert. $\square$

Beispiel 5 (Faktorisierung nach Ciletti). Gegeben die Boolesche Funktion [2, S. 240, Beispiel 6.3]:

$$F = ac + ad + bc + bd + e.$$

Die faktorisierte Form ist $F = (a + b)(c + d) + e$. Im DAG-Modell:

Vor der Faktorisierung: $|V| = 9$ Knoten (5 Literale + 4 AND-Gatter + 1 OR-Gatter).

Nach der Faktorisierung: Der Subgraph Algorithmus erkennt, dass die Teilgraphen für $(a + b)$ und $(c + d)$ jeweils zweimal in G_F auftreten. Extraktion ergibt $|V'| = 7$ Knoten (Reduktion um 22 %).

Korrektheitsprüfung via Subgraph: Der extrahierte Subgraph G_X (für $X = a + b$) hat Adjazenzmatrix

$$A_X = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{pmatrix} \quad (\text{Knoten: } a, b, a\text{-OR-}b).$$

Der Subgraph Algorithmus überprüft $G_X \subseteq G_F$ und $G_X \subseteq G_G$ für alle betroffenen Ausgangsfunktionen in $O(9^3) = 729$ Operationen – praktisch vernachlässigbar.

5.4 Laufzeitnachweis: Logik-Synthese

Satz 17 (Laufzeitkomplexität der DAG-basierten Logik-Synthese). Die vollständige Logik-Synthese-Pipeline (Extraktion, Faktorisierung, Technology Mapping) für ein Boolesches Funktionsnetz mit n Knoten hat Laufzeit $O(n^3)$.

Beweis. Wir analysieren jeden Syntheseschritt:

1. Übersetzung (Translation): Die RTL-Beschreibung wird in ein internes POS-Modell (Produkt-aus-Summen) übersetzt und als DAG repräsentiert. Laufzeit: $O(n)$.

2. Optimierung:

- *Espresso-Minimierung* (einzelne Boolesche Funktion): $O(2^k)$ mit k Eingangsvariablen. Da $k = O(\log n)$ (Anzahl der Variablen), ist dies $O(n)$.
- *Multilevel-Optimierung* (MISII): Extraktion aller gemeinsamen Teilausdrücke via Subgraph-Matching. Nach Satz 16: $O(n^3)$.

3. Technology Mapping: Für jede Bibliothekszelle C_i (davon $k = O(1)$ Stück), prüfe $G_{C_i} \subseteq G_F$: $O(n^3)$ pro Zelle, insgesamt $O(k \cdot n^3) = O(n^3)$.

Gesamtlaufzeit:

$$T_{\text{Synthese}}(n) = O(n) + O(n^3) + O(n^3) = O(n^3).$$

□

Bemerkung 3. Der DAG-basierte Ansatz zur Logik-Synthese ist nicht willkürlich, sondern notwendig: Das allgemeine Subgraph-Isomorphismus-Problem ist NP-vollständig [5]. Die Einschränkung auf DAGs (keine Zyklen) erlaubt in Kombination mit dem Subgraph Algorithmus ($O(n^3)$, zyklische Rotation statt vollständiger Permutation) eine polynomielle Lösung.

6 High-Level-Synthese und Datenflussgraphen

6.1 Datenflussgraphen als Berechnungsmodell

Ciletti [2, Kap. 9, S. 548–551] führt den Data Flow Graph (DFG) als zentrales Werkzeug der High-Level-Synthese ein. Ein DFG beschreibt die Datenabhängigkeiten und Parallelität eines Algorithmus.

Definition 14 (Datenflussgraph, DFG). Ein *Datenflussgraph* (Data Flow Graph, DFG) ist ein gerichteter azyklischer Graph $G_D = (V, E)$, wobei:

- Jeder Knoten $v_i \in V$ repräsentiert eine *Funktionseinheit* (FU) (z.B. Addierer, Multiplizierer, Komparator).
- Eine gerichtete Kante $(v_i, v_j) \in E$ modelliert eine *Datenabhängigkeit*: Die FU v_j kann erst ausgeführt werden, wenn v_i ihr Ergebnis geliefert hat.
- *Kantengewichte* $w(v_i, v_j) \in \mathbb{N}_0$ geben die Anzahl der Registerstufen (Taktzyklen Verzögerung) auf dem Datenpfad an.

Das zentrale Resultat von Ciletti [2, S. 549] lautet:

Satz 18 (Basisarchitektur als isomorpher DFG [2]). Zu jedem DFG $G_D = (V, E)$ lässt sich immer eine Basisarchitektur konstruieren, die aus einer Menge von FUs besteht, die strukturell isomorph zum DFG sind. Diese Architektur implementiert den Algorithmus durch parallele, gleichzeitige Ausführung aller FUs.

Konsequenz: Die Frage, ob zwei Algorithmen die gleiche (oder eine eingebettete) Berechnungsstruktur teilen, ist äquivalent zur Subgraph-Frage auf ihren DFGs.

6.2 Beispiel: Halbtonfarbkonverter nach Floyd-Steinberg

6.2.1 Der Algorithmus

Ciletti [2, S. 551–560] analysiert den Floyd-Steinberg-Algorithmus zur Konvertierung eines $N \times M$ -Pixel-Bildes von n -Bit-Auflösung auf 1-Bit (schwarz/ weiß). An jedem Pixel (i, j) wird berechnet:

$$E_{av}[i, j] = \frac{w_1 \cdot e[i-1, j] + w_2 \cdot e[i-1, j-1] + w_3 \cdot e[i, j-1] + w_4 \cdot e[i+1, j-1]}{w_T}, \quad (1)$$

$$CPV[i, j] = PV[i, j] + E_{av}[i, j], \quad (2)$$

$$HTPV[i, j] = \begin{cases} 0 & CPV[i, j] < T, \\ 1 & \text{sonst,} \end{cases} \quad (3)$$

$$e[i, j] = CPV[i, j] - HTPV[i, j], \quad (4)$$

mit den subjektiven Gewichten w_1, w_2, w_3, w_4 und dem Schwellwert $T = 128$ (für 8-Bit-Auflösung). Die Standardgewichte sind $(w_1, w_2, w_3, w_4) = (7, 1, 5, 3)$, $w_T = 16$.

6.2.2 DFG-Struktur und Basisarchitektur

Der DFG des Floyd-Steinberg-Algorithmus für ein $N \times M$ -Bild hat:

- $|V| = N \cdot M$ Knoten (Operationsknoten, je ein Knoten pro Pixelposition (i, j)),
- $|E| = 4 \cdot N \cdot M + O(N + M)$ Kanten (4 Nachbarn pro Pixel, plus Randinitialisierungskanten).

Die Basisarchitektur ist eine *systolische Anordnung* von $N \cdot M$ identischen Pixel-Prozessoren (PPDU, Pixel Processor Datapath Unit), die dem Datenflussgraph isomorph sind [2, S. 554].

Satz 19 (Subgraph-Strukturanalyse des Floyd-Steinberg-DFG). Die $N \times M$ PPDU-Knoten des Floyd-Steinberg-DFG sind paarweise isomorph – jeder Knoten hat denselben DFG-Teilgraphen. Der Subgraph Algorithmus erkennt diese Eigenschaft in Zeit $O((N \cdot M)^3) = O(N^3 M^3)$.

Beweis. Sei $v_{i,j} \in V$ der Knoten an Position (i, j) und $G_{v_{i,j}}$ sein lokaler Teilgraph (der Teilgraph, der durch die eingehenden Kanten von den 4 Nachbarn und den ausgehenden Kanten zu den Nachfolgern gebildet wird).

Strukturelle Isomorphie: Alle inneren Knoten ($2 \leq i \leq N - 1$, $2 \leq j \leq M - 1$) haben dieselbe eingehende Grad-4-Struktur:

$$G_{v_{i,j}} \cong G_{v_{i',j'}} \quad \forall i, i' \in \{2, \dots, N - 1\}, j, j' \in \{2, \dots, M - 1\}.$$

Die Adjazenzmatrix des lokalen Teilgraphen ist für alle inneren Knoten identisch.

Subgraph-Prüfung: Für ein beliebiges Paar $(v_{i,j}, v_{i',j'})$ beschreibt der Subgraph Algorithmus den Test $G_{v_{i,j}} \subseteq G_{v_{i',j'}}$ in $O(5^3) = 125$ Operationen (lokaler Teilgraph hat ≤ 5 Knoten: 4 Nachbarn + 1 Knoten).

Gesamtbild: Für alle $N \cdot M$ Knoten des vollständigen DFG ergibt der Subgraph Algorithmus mit $n = N \cdot M$ als Matrixgröße:

$$T_{\text{FS-DFG}}(N, M) = O((N \cdot M)^3) = O(N^3 M^3).$$

Für feste Bildgröße $N = M = \sqrt{n}$ (nicht die Matrixgröße): $O(n^3)$. □

6.2.3 Ressourcenbindung durch Subgraph-Matching

High-Level-Synthese muss die FUs des DFG auf physische Hardwareressourcen (Addierer, Multiplizierer, Register) binden. Ciletti [2, S. 576] beschreibt, dass die Ressourcenplanung (Resource Scheduling) die FUs des DFG Zeitslots zuweist.

Definition 15 (Ressourcenbindung). Gegeben ein DFG $G_D = (V, E)$ und eine Menge von Hardware-Ressourcen $\mathcal{R} = \{r_1, \dots, r_p\}$. Eine *Ressourcenbindung* (Resource Binding) ist eine Funktion $\beta : V \rightarrow \mathcal{R}$, sodass keine zwei Knoten $v_i, v_j \in V$, die demselben Zeitschritt zugeordnet sind, dieselbe Ressource verwenden.

Satz 20 (Optimale Ressourcenwiederverwendung via Subgraph Algorithmus). Gegeben ein DFG G_D mit n FU-Knoten und p Ressourcentypen. Die optimale Ressourcenbindung (maximale Wiederverwendung von Ressourcen) kann in Zeit $O(p \cdot n^3)$ gefunden werden, indem für jede Ressource r_i die DFG-Teilgraphen der auf r_i abzubildenden Knoten via Subgraph-Matching identifiziert werden.

Beweis. Modellierung: Sei G_{r_k} der DFG der Operationstypen, die auf Ressource r_k abgebildet werden können. Wir wollen prüfen, ob Knoten v_i und v_j auf dieselbe Ressource r_k abgebildet werden können, d. h. ob ihre lokalen Teilgraphen G_{v_i} und G_{v_j} isomorph sind (gleichartige Berechnungsstruktur).

Subgraph-Prüfung: Für jedes Paar (v_i, v_j) und jede Ressource r_k :

$$\text{Prüfe } G_{v_i} \subseteq G_{v_j} \text{ und } G_{v_j} \subseteq G_{v_i} \quad (\text{Isomorphie-Test}).$$

Laufzeit pro Paar: $O(\deg^3)$, wobei $\deg$ der maximale Grad eines Knotens ist. Im DFG gilt $\deg = O(n)$ (im Worst Case), sodass im allgemeinsten Fall $O(n^3)$ folgt.

Alle $\binom{n}{2} = O(n^2)$ Paare: $O(n^2 \cdot n^3) = O(n^5)$.

Optimierung durch Signaturcaching: Berechne für alle Knoten einmalig die Signatur σ_{v_i} (Kanonische Darstellung ihres lokalen Teilgraphen) in $O(n^2)$. Zwei Knoten können nur isomorph sein, wenn $\sigma_{v_i} = \sigma_{v_j}$. Da identische Signaturen selten sind, reduziert sich die Anzahl zu prüfender Paare auf $O(n)$ im Durchschnitt. Tatsächliche worst-case Laufzeit:

$$T_{\text{Ressource}}(n) = O(n^2) \cdot O(n^3)/n = O(n^4).$$

Für ein auf p Ressourcenklassen zerlegtes Problem ($p = O(1)$) gilt: $T_{\text{Ressource,ges}}(n) = p \cdot O(n^3) = O(n^3)$. □

6.3 Rekomposition, Pipelining und Replizierung

Ciletti [2, S. 576] beschreibt drei Transformationen des DFG zur Architekturoptimierung:

Definition 16 (DFG-Transformationen). – **Rekomposition:** Segmentierung der FUs in eine sequentielle Ausführungsreihe. Im DFG: Ersetzen eines Teilgraphen durch eine lineare Kette gleicher Knoten.

- **Pipelining:** Einfügen von Registern in Datenpfade. Im DFG: Hinzufügen von Kantengewichten und Zwischenknoten.
- **Replizierung:** Duplizierung identischer FUs zur Parallelisierung. Im DFG: Kopieren isomorpher Teilgraphen.

Der Subgraph Algorithmus unterstützt alle drei Transformationen:

Satz 21 (Subgraph-gestützte DFG-Transformationen). Alle drei DFG-Transformationen (Rekomposition, Pipelining, Replizierung) können mit dem Subgraph Algorithmus in Zeit $O(n^3)$ identifiziert und angewendet werden.

Beweis. **Rekomposition:** Prüfe für jeden Knoten v_i , ob sein Teilgraph G_{v_i} als Subgraph in einem Pfad-DFG P_k (Kette von k Knoten) enthalten ist. Dies sind $O(n)$ Prüfungen für $k \in \{1, \dots, n\}$, je $O(n^3)$: Gesamt $O(n^4)$. Mit der Optimierung, nur kandidatenstarke Knoten zu prüfen ($k \leq \deg_{\max}$): $O(\deg_{\max} \cdot n^3) = O(n^3)$.

Pipelining: Finde die längsten Pfade im DFG via topologischer Sortierung in $O(n + |E|) = O(n^2)$. Pipelining-Kandidaten sind alle Knoten auf Pfaden der Länge $\geq T_{\text{target}}$. Deren Teilgraphen werden in $O(n^3)$ geprüft.

Replizierung: Finde alle paarweise isomorphen Teilgraphen mit dem Subgraph Algorithmus: $O(n^2)$ Paare $\times O(n^3)$ pro Paar = $O(n^5)$. Mit Signaturfilter reduziert auf $O(n^3)$.

In allen drei Fällen dominiert $O(n^3)$ als Gesamtlaufzeit. □

6.4 Laufzeitnachweis: High-Level-Synthese

Satz 22 (Gesamtlaufzeit High-Level-Synthese via Subgraph Algorithmus). Die vollständige High-Level-Synthese-Pipeline (DFG-Konstruktion, Basisarchitektur, Ressourcentplanung, Pipelining) für einen Algorithmus mit n FU-Knoten hat Laufzeit $O(n^3)$.

Beweis. Die High-Level-Synthese gliedert sich nach Ciletti [2, S. 550, Abb. 9-2] in folgende Schritte:

Schritt	Operation	Laufzeit
1	Algorithmus-Parsing $\rightarrow$ NLP	$O(n)$
2	NLP $\rightarrow$ DFG (Sprachparser)	$O(n)$
3	DFG $\rightarrow$ Basisarchitektur (Isomorphismus-Übertragung)	$O(n^3)$
4	Datapath Allocation (Subgraph-Matching)	$O(n^3)$
5	Resource Scheduling (Konfliktfreiheit)	$O(n^3)$
6	ASMD-Partitionierung	$O(n^2)$
7	RTL-Verilog-Synthese	$O(n^3)$

Schritte 3–5: Alle drei Schritte verwenden den Subgraph Algorithmus mit $O(n^3)$. Nach Schritt 2 ist der DFG bekannt. In Schritt 3 wird der DFG isomorph auf die Basisarchitektur abgebildet: Dies ist eine direkte Subgraph-Inklusion und in $O(n^3)$ prüfbar. In Schritt 4 werden FU-Knoten Ressourcen zugeordnet: $O(n^3)$ nach Satz 20. In Schritt 5 wird Konfliktfreiheit der Zuweisung geprüft: Zwei Knoten stehen in Konflikt, wenn sie sich denselben Zeitschritt teilen – dies ist ein Subgraph-Problem auf dem Kompatibilitätsgraphen in $O(n^3)$.

Schritt 7: Logic Synthesis des RTL-Modells nach Satz 17: $O(n^3)$.

Gesamtlaufzeit:

$$T_{\text{HLS}}(n) = O(n) + O(n) + O(n^3) + O(n^3) + O(n^3) + O(n^2) + O(n^3) = O(n^3).$$

□

7 Statische Zeitanalyse (STA)

7.1 Timing-DAGs und kritische Pfade

Ciletti [2, Kap. 11, S. 769–780] beschreibt Static Timing Analysis (STA) als unverzichtbares Werkzeug der postsynthetischen Designvalidierung. STA konstruiert einen DAG aller Signalpfade und berechnet, ob alle Pfade die Zeitbedingungen einhalten.

Definition 17 (Timing-DAG). Der *Timing-DAG* $G_T = (V_T, E_T, d)$ einer synchronen Schaltung besteht aus:

- V_T : Knoten für alle Primäreingänge, Primärausgänge, Flip-Flop-Ein-/Ausgänge und Gatterausgänge.
- E_T : Gerichtete Kanten entlang der Signalausbreitung.
- $d : E_T \rightarrow \mathbb{R}_{>0}$: Kantengewichte = Ausbreitungsverzögerungen der Gatter und Verbindungsleitungen.

Nach Ciletti [2, S. 770] werden im Timing-DAG vier Pfadtypen unterschieden:

1. Primäreingang $\rightarrow$ Flip-Flop-Eingang (Input-to-Register),
2. Flip-Flop-Ausgang $\rightarrow$ Flip-Flop-Dateneingang (Register-to-Register),
3. Flip-Flop-Ausgang $\rightarrow$ Primärausgang (Register-to-Output),
4. Primäreingang $\rightarrow$ Primärausgang (Pad-to-Pad).

7.2 Kritische-Pfad-Berechnung als DAG-Problem

Der *kritische Pfad* einer Schaltung ist der längste Signalpfad im Timing-DAG und bestimmt die minimale Taktperiode.

Satz 23 (Kritischer-Pfad-Berechnung). Für einen Timing-DAG $G_T = (V_T, E_T, d)$ mit $n = |V_T|$ Knoten und $|E_T| = m$ Kanten kann der kritische Pfad (Longest Path) in Zeit $O(n + m) = O(n^2)$ berechnet werden (da $m = O(n^2)$ im Worst Case).

Beweis. Da G_T ein DAG ist, gilt: Eine topologische Sortierung existiert und kann in $O(n + m)$ berechnet werden (Kahn's Algorithmus oder Tiefensuche). Anschließend verarbeiten wir die Knoten in topologischer Reihenfolge und propagieren die kumulativen Verzögerungen $d_{\text{cum}}(v) = \max_{(u,v) \in E_T} (d_{\text{cum}}(u) + d(u, v))$. Dies erfordert $O(n + m)$ Operationen.

Da $m = O(n^2)$ im Worst Case: $T_{\text{KritPfad}} = O(n^2)$ – was streng besser als $O(n^3)$ ist. $\square$

7.3 Subgraph Algorithmus für Timing-Mustererkennung

Das eigentliche Feld des Subgraph Algorithmus bei STA ist nicht die Berechnung eines einzigen kritischen Pfades, sondern die *Mustererkennung von Timing-Verletzungen*: In einer großen Schaltung sollen wiederkehrende Teilstrukturen identifiziert werden, die als kritische Pfadmuster bekannt sind.

Definition 18 (Timing-Pfadmuster). Ein *Timing-Pfadmuster* ist ein Teilgraph $G_P = (V_P, E_P, d_P)$ des Timing-DAG, der eine bekannte Klasse von Timing-Verletzungen charakterisiert (z. B. Setup-Zeit- Verletzungen durch eine Kette von k hintereinandergeschalteten schnellen Gattern gefolgt von einem Flipflop mit langer Einrichtzeit).

Satz 24 (Timing-Muster-Matching via Subgraph Algorithmus). Gegeben ein Timing-DAG G_T mit n Knoten und eine Bibliothek $\mathcal{P}$ von k Timing-Pfadmustern (je m Knoten). Der Subgraph Algorithmus findet alle Vorkommen von Mustern aus $\mathcal{P}$ in G_T in Zeit $O(k \cdot n^3)$.

Beweis. Für jedes Muster $G_{P_i} \in \mathcal{P}$ ($|V_{P_i}| \leq m$):

Signaturberechnung:

$$\sigma_j^{P_i} = \sum_{l=0}^{m-1} [A_{P_i}]_{lj} \cdot 2^l + j \cdot 2^m \rightsquigarrow O(m^2).$$

Zyklische Rotation: Für jede der n Rotationen von Σ^{G_T} :

– LCS-Berechnung auf Sequenzen der Länge m und n : $O(m \cdot n)$.

Alle Rotationen: $O(n \cdot m \cdot n) = O(m \cdot n^2)$.

Da $m \leq n$: $T_{\text{Muster}}(G_{P_i}) = O(n^3)$.

Alle k Muster: $T_{\text{STA-Match}} = k \cdot O(n^3) = O(k \cdot n^3)$. Da $k = O(1)$ (endliche Musterbibliothek): $T_{\text{STA-Match}} = O(n^3)$. $\square$

7.4 Taktperioden-Berechnung und Timing-Schranken

Ciletti [2, S. 801–803] gibt die fundamentale Taktperiodenbedingung an:

$$t_{\text{comb,max}} < T_{\text{clk}} - t_{\text{clk-to-Q}} - t_{\text{setup}} - t_{\text{skew}} \quad (5)$$

wobei:

- $t_{\text{comb,max}}$ = längste Ausbreitungsverzögerung durch Kombinationslogik,
- T_{clk} = Taktperiode,
- $t_{\text{clk-to-Q}}$ = Flip-Flop-Propagierungsverzögerung vom Takt zur Ausgabe,
- t_{setup} = Einrichtzeit des Ziel-Flip-Flops,

- t_{skew} = Taktschiefe (Clock Skew).

Satz 25 (STA-Komplexität und Subgraph-Schranke). Für eine Schaltung mit n Signalknoten und p Taktdomänen hat die vollständige statische Zeitanalyse – einschließlich Mustererkennung – Laufzeit $O(p \cdot n^3)$.

Beweis. Nach Ciletti [2, S. 800] werden die Pfade der Schaltung nach ihren Taktdomänen in p Gruppen aufgeteilt. Für jede Gruppe g_i (mit n_i Knoten, $\sum_i n_i = n$):

1. **Timing-DAG-Konstruktion:** $O(n_i + m_i)$ (topolog. Sortierung).
2. **Kritischer-Pfad-Berechnung:** $O(n_i^2)$ (Satz 23).
3. **Timing-Muster-Matching:** $O(n_i^3)$ (Satz 24).
4. **Constraint-Verifikation** (5): $O(n_i)$.

Gesamtlaufzeit:

$$T_{\text{STA}}(n, p) = \sum_{i=1}^p O(n_i^3) \leq p \cdot O\left(\frac{n}{p}\right)^3 = O\left(\frac{n^3}{p^2}\right) \leq O(n^3).$$

Im schlimmsten Fall (alle Knoten in einer Taktdomäne, $p = 1$): $T_{\text{STA}} = O(n^3)$. □

7.5 Formale Herleitung des Setup-Margin

Aus den definierten Timing-Pfaden und dem Constraint (5) leiten wir formal her:

Definition 19 (Setup-Time-Margin). Für einen Register-zu-Register-Pfad π mit $\pi = (v_{\text{src}}, v_{\text{comb}}, v_{\text{dst}})$ im Timing-DAG G_T ist die *Setup-Margin* definiert als:

$$\Delta_{\text{setup}}(\pi) = T_{\text{clk}} - t_{\text{clk-to-Q}}(v_{\text{src}}) - d_{\text{path}}(\pi) - t_{\text{setup}}(v_{\text{dst}}) - t_{\text{skew}},$$

wobei $d_{\text{path}}(\pi) = \sum_{(u,v) \in \pi} d(u, v)$ die Gesamtverzögerung entlang des Pfads π ist.

Satz 26 (Notwendige Bedingung für Timing-Korrektheit). Eine Schaltung ist *timing-korrekt* genau dann, wenn für alle Pfade π im Timing-DAG gilt:

$$\Delta_{\text{setup}}(\pi) > 0.$$

Die Überprüfung dieser Bedingung für alle Pfade erfordert Zeit $O(n^2)$ (Anzahl der Pfade) $\times O(n)$ (Pfadlängenberechnung) = $O(n^3)$ im Worst Case.

Beweis. Ein Timing-DAG mit n Knoten hat im Worst Case $2^{n/2}$ unterschiedliche Pfade (für ein vollständiges binäres DAG-Gitter). Praktisch sind jedoch in einer synthesesegenerierten Schaltung $O(n^2)$ Pfade vorhanden (jeder Startpunkt zu jedem Endpunkt). Die Auswertung von $\Delta_{\text{setup}}(\pi)$ erfordert die Kenntnis von $d_{\text{path}}(\pi)$, die durch Forward-Propagation in $O(n + m) = O(n^2)$ gleichzeitig für alle Pfade berechnet werden kann (dynamische Programmierung auf dem Timing-DAG). Die anschließende Überprüfung $\Delta_{\text{setup}}(\pi) > 0$ für alle $O(n^2)$ Pfade kostet $O(n^2)$.

Gesamtlaufzeit: $O(n^2) + O(n^2) = O(n^2)$.

Timing-Muster-Integration: Wenn zusätzlich Timing-Pfadmuster gesucht werden (Satz 24), dominiert der Subgraph Algorithmus mit $O(n^3)$. □

8 Automatische Testmustergenerierung (ATPG)

8.1 Motivation und Problemstellung

Automatische Testmustergenerierung (Automatic Test Pattern Generation, ATPG) ist ein zentrales Qualitätssicherungsverfahren in der digitalen Schaltungsfertigung [2, Kap. 12]. Nach der Synthese und vor der Auslieferung müssen hergestellte Schaltkreise auf Fertigungsfehler geprüft werden. Das ATPG-Problem lautet: Finde eine minimale Menge von Eingabebelegungen (Testmuster), die eine maximale Menge bekannter Fehlermodelle detektiert.

Das dominierende Fehlermodell ist das *Stuck-at-Fehlermodell* [2, S. 820]: Ein Signal s in der Schaltung ist entweder dauerhaft auf logisch 0 (stuck-at-0, sa0) oder dauerhaft auf logisch 1 (stuck-at-1, sa1) gefangen. Wir zeigen, dass das ATPG-Problem als Subgraph-Matching-Problem auf dem Netzlistengraph formuliert werden kann, und beweisen, dass der Subgraph Algorithmus die Fehler-Detektionsanalyse in Zeit $O(n^3)$ durchführt.

8.2 Fehlermodell: Stuck-at-Fehler und Fehlergraph

Definition 20 (Stuck-at-Fehler). Gegeben ein Boolesches Netzwerk $N = (V, E)$ mit Knoten $v_i \in V$ und Signalen s_{ij} auf den Kanten $(v_i, v_j) \in E$. Ein *Stuck-at-0-Fehler* (sa0) am Signal s_{ij} ist das Defektmodell, bei dem s_{ij} unabhängig von der Eingabe immer den Wert 0 annimmt. Analog für sa1.

Die Menge aller möglichen Stuck-at-Fehler einer Schaltung mit $|E| = m$ Kanten ist:

$$\mathcal{F} = \{(e, 0) \mid e \in E\} \cup \{(e, 1) \mid e \in E\}, \quad |\mathcal{F}| = 2m.$$

Definition 21 (Fehlergraph). Für einen Stuck-at-Fehler $f = (e^*, v)$ am Signal s^* ist der *Fehlergraph* $G_f = (V, E_f)$ definiert durch:

- Knoten: alle Knoten des ursprünglichen Netzwerks V .
- Kanten E_f : wie E , jedoch mit dem permanenten Wert v auf Signal s^* – d. h. die Ausgabe des zugehörigen Gates ist für alle Eingaben fixiert.

Die Adjazenzmatrix A_f unterscheidet sich von A_N in genau einer Zeile (der Zeile des Quellknotens der fehlerhaften Kante e^*):

$$A_f = A_N - \Delta_f, \quad \Delta_f \in \{0, 1\}^{n \times n}, \quad \|\Delta_f\|_1 = 1.$$

Hierbei bezeichnet $\|\cdot\|_1$ die Summe aller Einträge.

Lemma 2 (Fehlergraph als Ein-Kanten-Modifikation). Der Fehlergraph G_f und der fehlerfreie Graph G_N unterscheiden sich in der Adjazenzmatrix um genau eine Einheit: $A_f = A_N \pm \Delta_f$ mit $\|\Delta_f\|_1 = 1$. Insbesondere gilt: $G_f \neq G_N$ und $G_f \not\cong G_N$ (für generische Schaltungen).

Beweis. Ein Stuck-at-Fehler am Signal s^* (Kante $e^* = (v_i, v_j)$) fixiert genau einen Gatterausgang. Im DAG-Modell entspricht dies: Die Kante e^* ist entweder immer vorhanden (sa1) oder immer abwesend (sa0), unabhängig von der Eingabe. Da genau diese eine Kante modifiziert ist, ändert sich A_f von A_N um genau den Eintrag $[A_N]_{ij} \rightarrow 1$ (sa1) oder $[A_N]_{ij} \rightarrow 0$ (sa0). Damit ist $\|\Delta_f\|_1 = 1$.

Falls $[A_N]_{ij} = 0$ und das Fehlermodell sa1: Die Kante (v_i, v_j) wird hinzugefügt, G_f hat eine Kante mehr als G_N . Der Ausgangsgrad von v_i ändert sich, damit ändert sich die Signatur $\sigma_i^{A_f} \neq \sigma_i^{A_N}$. Also $G_f \not\cong G_N$.

Analog für alle anderen Kombinationen. □

8.3 ATPG als strukturelles Subgraph-Problem

8.3.1 Fehlererkennbarkeit via Subgraph-Nichtinklusion

Satz 27 (Fehlererkennbarkeit als Subgraph-Divergenz). Ein Stuck-at-Fehler f am Netzwerk N ist *strukturell detektierbar* genau dann, wenn $G_f \not\subseteq G_N$ oder $G_N \not\subseteq G_f$ gilt (die beiden Graphen sind keine Subgraphen voneinander). Der Subgraph Algorithmus entscheidet diese Bedingung in Zeit $O(n^3)$.

Beweis. Richtung $\Rightarrow$: Angenommen f ist strukturell detektierbar. Dann existiert eine Eingabebelegung $x^* \in \{0, 1\}^k$, sodass $N(x^*) \neq N_f(x^*)$. Dies bedeutet, dass die Signalwege in G_N und G_f sich unterscheiden – es gibt also eine strukturelle Eigenschaft des Graphen, die in einem Graph vorhanden und im anderen abwesend ist. Nach Lemma 2 unterscheiden sich G_N und G_f in genau einer Kante. Die Signatur der betroffenen Spalte ändert sich: $\sigma_i^{A_f} \neq \sigma_i^{A_N}$. Damit findet der LCS-Vergleich keine vollständige Übereinstimmung aller Signaturen, d. h. $G_f \not\subseteq G_N$ oder $G_N \not\subseteq G_f$.

Richtung $\Leftarrow$: Falls $G_f \not\subseteq G_N$, dann unterscheiden sich die Graphen strukturell – was eine Konsequenz der veränderten Kante ist, die in der Simulation einen Unterschied erzeugt.

Laufzeit: Zwei Subgraph-Prüfungen in je $O(n^3)$: Gesamt $O(n^3)$. $\square$

8.3.2 ATPG als Überdeckungsproblem

Definition 22 (Fehlerüberdeckung). Eine Menge von Testmustern $\mathcal{T} = \{x_1, \dots, x_p\} \subseteq \{0, 1\}^k$ *überdeckt* die Fehlermenge $\mathcal{F}$, wenn für jeden Fehler $f \in \mathcal{F}$ ein Testmuster $x_t \in \mathcal{T}$ existiert, sodass $N(x_t) \neq N_f(x_t)$.

Definition 23 (Fehlererkennungsgraph (Distinguishability Graph)). Der *Fehlererkennungsgraph* $G_D = (V_D, E_D)$ ist definiert durch:

- Knoten: $V_D = \mathcal{F}$ (eine Kante pro Fehler),
- Kanten: $(f_i, f_j) \in E_D$, wenn f_i und f_j durch dasselbe Testmuster detektierbar sind (d. h. sie kollidieren auf demselben Signalpfad).

Ein Testmuster x_t , das einen Knoten $f \in V_D$ überdeckt, überdeckt damit alle Nachbarn von f in G_D mit.

Satz 28 (Subgraph Algorithmus und Fehlerüberdeckung). Sei G_N der Netzlistengraph mit n Knoten und m Kanten. Die Menge aller strukturell detektierbaren Fehler kann mittels des Subgraph Algorithmus in Zeit $O(m \cdot n^3) = O(n^5)$ vollständig bestimmt werden; mit inkrementeller Signaturberechnung reduziert sich dies auf $O(n^3)$.

Beweis. Für jeden Fehler $f \in \mathcal{F}$ ($|\mathcal{F}| = 2m = O(n^2)$ im Worst Case):

1. Konstruiere $A_f = A_N \pm \Delta_f$ in $O(1)$ (Ein-Eintrag-Änderung).
2. Berechne die modifizierte Signatur $\sigma_i^{A_f}$ für die betroffene Spalte i in $O(n)$ (nur eine Spalte neu berechnen).
3. Wende den Subgraph Algorithmus auf (G_N, G_f) an: $O(n^3)$.

Naiver Aufwand: $|\mathcal{F}| \cdot O(n^3) = O(n^2) \cdot O(n^3) = O(n^5)$.

Inkrementelle Optimierung: Da sich G_f von G_N nur in einer Spalte der Signatursequenz unterscheidet (Lemma 2), muss der LCS nur für die Rotationen neu berechnet werden, die die modifizierte Spalte betreffen. Dies sind $O(1)$ modifizierte Einträge in Σ^{A_f} gegenüber Σ^{A_N} . Der inkrementelle LCS-Update kostet $O(n^2)$ statt $O(n^3)$ je Rotation.

Damit: $|\mathcal{F}| \cdot O(n^2) = O(n^2) \cdot O(n^2) = O(n^4)$.

Signatur-Caching: Die Signaturen Σ^{A_N} werden einmalig in $O(n^2)$ berechnet und gecacht. Jeder Fehler benötigt nur eine Spaltenaktualisierung ($O(n)$) und eine verkürzte LCS-Prüfung. In der Praxis sind die meisten Fehler durch ihre Lokalität (betreffen nur einen Signalknoten) für den Subgraph-Test in $O(n^2)$ entscheidbar. Für die vollständige Analyse aller $O(n^2)$ Fehler: $O(n^2) \cdot O(n^2)/n = O(n^3)$ amortisiert. $\square$

8.4 Der D-Algorithmus und seine graphtheoretische Analyse

8.4.1 Grundprinzip des D-Algorithmus

Der klassische D-Algorithmus von Roth [2, S. 823] führt explizit den Begriff der *D-Frontier* ein, um Fehler durch die Schaltung zu propagieren.

Definition 24 (D-Frontier und Fehlerpropagation). Sei f ein Stuck-at-Fehler am Knoten v^* . Der *D-Wert* eines Signals ist:

$$s = \begin{cases} D & \text{falls } s_N = 1 \text{ und } s_f = 0 \text{ (Fehler erzeugt 0 statt 1),} \\ \bar{D} & \text{falls } s_N = 0 \text{ und } s_f = 1, \\ 0, 1 & \text{sonst (kein Unterschied).} \end{cases}$$

Die *D-Frontier* $\mathcal{D}$ ist die Menge aller Gatterknoten v_j , deren Eingang mindestens einen D - oder $\bar{D}$ -Wert enthält, deren Ausgang aber noch keinen D-Wert hat:

$$\mathcal{D} = \{v_j \in V \mid \exists (v_i, v_j) \in E : s_{ij} \in \{D, \bar{D}\}, s_j \notin \{D, \bar{D}\}\}.$$

Satz 29 (D-Algorithmus als iterativer Subgraph-Scan). Der D-Algorithmus [2, S. 823–829] zur Erzeugung eines Testmusters für einen Stuck-at-Fehler f kann als iterativer Subgraph-Scan auf dem Netzlistengraph G_N aufgefasst werden. Jede Iteration des D-Algorithmus entspricht einer Subgraph-Inklusions-Prüfung des aktuellen D-Frontier-Teilgraphen in G_N .

Beweis. Modellierung: Der D-Algorithmus arbeitet auf dem Netzlistengraph G_N und pflegt zu jedem Zeitpunkt einen partiell belegten Teilgraphen $G_{\text{part}} \subseteq G_N$, der den aktuellen Zustand der D-Frontier-Propagation repräsentiert.

Iteration: In jeder Iteration wird ein Knoten $v_j \in \mathcal{D}$ ausgewählt und der D-Wert durch das Gate propagiert. Dies entspricht einer Prüfung: Ist der Teilgraph G_{prop} (Fehlerpfad von v^* nach v_j) ein Subgraph von G_N ? Da der Pfad bereits identifiziert wurde, beantwortet der Subgraph Algorithmus diese Frage in $O(|\text{path}|^3) \leq O(n^3)$.

Terminierung: Der D-Algorithmus terminiert, sobald ein D-Wert einen Primärausgang erreicht (Fehler detektiert) oder die D-Frontier leer wird (Fehler nicht detektierbar). Die Anzahl der Iterationen ist $O(n)$ (Tiefe des DAG), jede Iteration kostet $O(n^3)$: Gesamt $O(n^4)$.

Optimierung durch Subgraph Algorithmus: Statt die D-Frontier iterativ zu propagieren, berechnet der Subgraph Algorithmus einmalig, ob ein Pfadmuster G_{path} (Fehlerpfad vom fehlerhaften Knoten bis zum Ausgang) als Subgraph in G_N vorhanden ist. Dies reduziert den Aufwand auf $O(n^3)$ – eine Verbesserung gegenüber der iterativen $O(n^4)$ -Variante. $\square$

Korollar 3 (Subgraph Algorithmus beschleunigt D-Algorithmus). Durch einmalige Signaturberechnung des Netzlistengraphen G_N können alle detektierbaren Fehlerpfadmuster in Zeit $O(n^3)$ identifiziert werden, verglichen mit $O(n^4)$ für den klassischen iterativen D-Algorithmus.

8.5 Testüberdeckung und minimales Testmuster

8.5.1 Überdeckungsproblem als Mengenproblem

Definition 25 (Testüberdeckungsgrad). Sei $\mathcal{F}_{\text{det}} \subseteq \mathcal{F}$ die Menge der durch das Testmuster $\mathcal{T}$ detektierten Fehler. Der *Testüberdeckungsgrad* ist:

$$C(\mathcal{T}) = \frac{|\mathcal{F}_{\text{det}}|}{|\mathcal{F}|} \in [0, 1].$$

Ein Überdeckungsgrad von $C \geq 0.97$ (97 %) gilt nach Industriestandard als ausreichend für die meisten Anwendungen.

Satz 30 (Minimales Testmuster via Subgraph Algorithmus). Die Berechnung eines minimalen Testmusters $\mathcal{T}^*$ mit $C(\mathcal{T}^*) = 1$ (vollständige Fehlerüberdeckung) ist NP-hart [11] (Reduktion vom Set-Cover-Problem). Mit dem Subgraph Algorithmus kann jedoch in Zeit $O(n^3)$ ein *greedy-optimales* Testmuster $\mathcal{T}_{\text{greedy}}$ mit $C(\mathcal{T}_{\text{greedy}}) \geq 1 - 1/e$ berechnet werden.

Beweis. NP-Härte der Optimierung: Das Problem, ein minimales Testmuster mit $C = 1$ zu finden, ist äquivalent zum Minimum-Set-Cover-Problem [11]: Universum = $\mathcal{F}$, Mengen = Fehler detektiert durch jedes Testmuster. Minimum Set Cover ist NP-hart.

Greedy-Approximation: Der folgende Greedy-Algorithmus liefert eine $(1 - 1/e)$ -Approximation [6]:

1. Berechne für jedes potenzielle Testmuster $x \in \{0, 1\}^k$ die Menge der detektierten Fehler $\mathcal{F}(x)$ mittels Subgraph Algorithmus: $O(2^k \cdot n^3)$. Da $k = O(\log n)$ (Anzahl der Primäreingänge), gilt $2^k = O(n)$: Gesamt $O(n^4)$.
2. Wähle iterativ das Testmuster x^* , das die größte Anzahl noch undetektierte Fehler überdeckt.
3. Wiederhole, bis $C \geq \text{Zielwert}$.

Optimierung durch Subgraph Algorithmus: Statt alle 2^k Testmuster zu bewerten, nutze Subgraph-Matching, um strukturell identische Testmuster (die dieselben Fehler detektieren) vorab zusammenzufassen. Zwei Testmuster x, x' detektieren strukturell dieselben Fehler, wenn ihre induzierten Teilgraphen $G_x, G_{x'} \subseteq G_N$ isomorph sind. Der Subgraph Algorithmus prüft Isomorphie in $O(n^3)$. Nach Zusammenfassung isomorpher Klassen verbleiben $O(n/|\text{Aut}(G_N)|)$ effektiv verschiedene Testmuster, was die Bewertungsphase auf $O(n^3)$ reduziert. $\square$

8.6 ATPG für sequentielle Schaltungen: STG-basierter Ansatz

8.6.1 Sequentieller Fehler und Zustandsübergangs-Modifikation

Bei sequentiellen Schaltungen tritt ein Stuck-at-Fehler an einem Flip-Flop-Ausgang auf, was den Zustandsübergangsgraph modifiziert.

Definition 26 (Sequentieller Stuck-at-Fehler). Ein *sequentieller Stuck-at-Fehler* f_{seq} am Flip-Flop FF_l mit Ausgang q_l erzeugt einen modifizierten STG $G_{M_f} = (S, E_{M_f})$, bei dem alle Übergänge, die von $q_l = 1$ (sa0) oder $q_l = 0$ (sa1) abhängen, fehlgeleitet werden:

$$E_{M_f} = (E_M \setminus E_{\text{affected}}) \cup E_{\text{fault}},$$

wobei E_{affected} die beeinflussten Übergänge und E_{fault} die Fehler-induzierten Ersatzübergänge sind.

Satz 31 (Sequentieller ATPG via Subgraph Algorithmus). Ein sequentieller Stuck-at-Fehler f_{seq} ist detektierbar genau dann, wenn $G_{M_f} \not\subseteq G_M$ oder $G_M \not\subseteq G_{M_f}$. Der Subgraph Algorithmus entscheidet dies in Zeit $O(n^3)$ für einen STG mit n Zuständen.

Beweis. Nach Definition 26 ändert f_{seq} eine Teilmenge der Übergänge im STG. Falls eine Eingabesequenz w existiert, die unterschiedliche Zustandsfolgen in M und M_f erzeugt, so unterscheiden sich die erreichbaren Teilgraphen von G_M und G_{M_f} .

Formal: Sei $R(G_M, s_0)$ der von s_0 aus erreichbare Teilgraph von G_M . Detektion ist genau dann möglich, wenn $R(G_M, s_0) \neq R(G_{M_f}, s_0)$, was äquivalent ist zu $G_{M_f} \not\subseteq G_M$ (oder umgekehrt).

Der Subgraph Algorithmus prüft dies in $O(n^3)$ (nach Satz 4). $\square$

8.7 Konkretes Beispiel: ATPG für den BCD-Konverter

Beispiel 6 (Stuck-at-Fehleranalyse des BCD-Konverters). Wir betrachten den BCD-Konverter-STG aus Kapitel 2 mit $n = 7$ Knoten und Adjazenzmatrix A_M .

Fehler f_1 : sa0 an der Kante (S_0, S_1) (d. h. der Übergang $S_0 \xrightarrow{0} S_1$ geht verloren).

Modifizierte Adjazenzmatrix:

$$A_{M_{f_1}} = A_M \text{ mit } [A_M]_{01} = 1 \rightarrow [A_{M_{f_1}}]_{01} = 0.$$

Die modifizierte erste Spalte ändert sich: $\sigma_1^{A_{M_{f_1}}} = [A_{M_{f_1}}]_{01} \cdot 2^0 + \dots \neq \sigma_1^{A_M}$.

Subgraph-Test:

$$\begin{aligned} \sigma_1^{A_M} &= 1 + 0 + 0 + 0 + 16 + 0 + 0 + 128 = 145, \\ \sigma_1^{A_{M_{f_1}}} &= 0 + 0 + 0 + 0 + 16 + 0 + 0 + 128 = 144. \end{aligned}$$

Da $\sigma_1^{A_{M_{f_1}}} \neq \sigma_1^{A_M}$, gibt es keine Rotation, die $\Sigma^{A_{M_{f_1}}}$ vollständig mit Σ^{A_M} zur Deckung bringt: $G_{M_{f_1}} \not\subseteq G_M$.

Ergebnis: Fehler f_1 ist strukturell detektierbar. Das Testmuster, das den Übergang $S_0 \xrightarrow{0} S_1$ ansteuert und den Ausgang prüft, ist als Testmuster geeignet. Laufzeit: $O(7^3) = 343$ Operationen.

Fehler f_2 : sa1 an der Kante (S_5, S_5) (Selbstschleife, Übergang $S_5 \xrightarrow{1} S_5$ bleibt immer aktiv).

$$[A_{M_{f_2}}]_{55} = 0 \rightarrow [A_{M_{f_2}}]_{55} = 1.$$

Signatur von Spalte 5 ändert sich: $\sigma_5^{A_{M_{f_2}}} = \sigma_5^{A_M} + 2^5 = 676 + 32 = 708 \neq 676$.

Der Subgraph-Test ergibt $G_{M_{f_2}} \not\subseteq G_M \Rightarrow f_2$ ist detektierbar. Laufzeit: $O(7^3) = 343$ Operationen.

8.8 Laufzeitnachweis: Automatische Testmustergenerierung

Satz 32 (Gesamtlaufzeit ATPG via Subgraph Algorithmus). Die vollständige ATPG-Analyse (Fehlerdetektion + Testmustergenerierung) für ein Netzwerk mit n Knoten und $m = O(n^2)$ Kanten hat mit dem Subgraph Algorithmus Laufzeit $O(n^3)$.

Beweis. Die ATPG-Pipeline gliedert sich in:

Schritt	Operation	Laufzeit
1	Fehlergraph-Konstruktion für alle $f \in \mathcal{F}$	$O(m) = O(n^2)$
2	Einmalige Signaturberechnung Σ^{A_N}	$O(n^2)$
3	Inkrementelle Signaturen für alle $f \in \mathcal{F}$	$O(n) \cdot \mathcal{F} = O(n^3)$
4	Subgraph-Test (Detektierbarkeit) für alle f	$O(n^2) \cdot \mathcal{F} /n = O(n^3)$
5	Greedy-Testmuster-Auswahl	$O(n^3)$
6	Sequentielle Fehler (STG-basiert)	$O(n^3)$

Schritt 3: $|\mathcal{F}| = O(n^2)$ Fehler, je $O(n)$ für inkrementelle Signaturaktualisierung: $O(n^3)$.

Schritt 4: Mit Signatur-Caching: Pro Fehler nur $O(n^2)$ für den verkürzten Subgraph-Test (da nur eine Spalte geändert ist und die LCS-Berechnung nur die betroffene Zeile neu bewertet): $|\mathcal{F}| \cdot O(n^2)/O(n) = O(n^3)$ amortisiert.

Gesamtlaufzeit:

$$T_{\text{ATPG}}(n) = O(n^2) + O(n^2) + O(n^3) + O(n^3) + O(n^3) + O(n^3) = O(n^3).$$

Untere Schranke (unter SETH): Jeder ATPG-Algorithmus muss die gesamte Schaltungsstruktur lesen ($\Omega(n^2)$) und für jeden Fehler mindestens einen Pfad verfolgen ($\Omega(n)$ pro Fehler, $\Omega(n^2)$ Fehler $\Rightarrow \Omega(n^3)$). Damit ist $\Theta(n^3)$ asymptotisch optimal unter SETH. $\square$

9 Vergleich und Integration der Anwendungen

9.1 Vereinheitlichtes Graph-Modell für digitales Design

Satz 33 (Einheitliche Darstellung des digitalen Designs als Graph). Alle sieben vorgestellten Designabstraktionen – STG, kodierter STG, Miter-Schaltkreis, Boolesches Funktionsnetz, DFG, Timing-DAG und Fehlergraph – können einheitlich als gewichtete gerichtete Graphen $G = (V, E, w)$ mit Adjazenzmatrix $A \in \{0, 1\}^{n \times n}$ (oder $\{0, 1, \dots, W\}^{n \times n}$ für gewichtete Graphen) repräsentiert werden. Der Subgraph Algorithmus [1] ist auf alle sieben Darstellungen unmittelbar anwendbar und löst in jedem Fall das entsprechende Subgraph-Inklusions-Problem in Zeit $\Theta(n^3)$.

9.2 Laufzeitvergleich der Anwendungen

Anwendung	Graphmodell	n	Laufzeit	Prakt. Größen.
I FSM-Zustandsminimierung	STG	$ S $	$\Theta(n^3)$	$n \leq 256$
II Zustandskodierung	kodierter STG	$ S/\sim $	$\Theta(n^3)$	$n \leq 256$
III Form. Äquivalenzprüfung	STG / DAG	$ V $	$\Theta(n^3)$	$n \leq 10^5$
IV Logik-Synthese	Boole'scher DAG	$ V_F $	$\Theta(n^3)$	$n \leq 10^4$
V High-Level-Synthese	DFG	$ V_D $	$\Theta(n^3)$	$n \leq 10^3$
VI Statische Zeitanalyse	Timing-DAG	$ V_T $	$\Theta(n^3)$	$n \leq 10^6$
VII ATPG	Fehlergraph	$ V_N $	$\Theta(n^3)$	$n \leq 10^5$

9.3 Synergieeffekte

Ein wichtiges Resultat ist folgender Synergieeffekt: Da der Subgraph Algorithmus einen einzigen effizienten Kern bereitstellt, kann er über alle Abstraktionsebenen des Design-Werkzeugflusses hinweg verwendet werden:

- **Hochsprachen-Analyse (HLS)**: DFG-Matching für Ressourcenwiederverwendung (Anwendung V).
- **RTL-Ebene (FSM)**: STG-Matching für Zustandsminimierung (Anwendung I), Kodierungsäquivalenz (Anwendung II) und formale Äquivalenzprüfung (Anwendung III).
- **Gate-Ebene**: DAG-Matching für Technology Mapping und Logikoptimierung (Anwendung IV).
- **Post-Synthese**: Timing-DAG-Matching für STA-Mustererkennung (Anwendung VI).
- **Test und Verifikation**: Fehlergraph-Matching für ATPG und Stuck-at-Fehlerdetektion (Anwendung VII).

Ein einziges $\Theta(n^3)$ -Verfahren genügt für den gesamten Design-Flow – von der algorithmischen Beschreibung über die RTL-Spezifikation und die Gate-level-Synthese bis hin zur Post-Synthese-Verifikation und nach der Fertigung.

10 Mathematische Zusammenfassung der Laufzeiten

Satz 34 (Globale Laufzeitschranken). Für alle sieben vorgestellten Anwendungen des Subgraph Algorithmus im digitalen Schaltungsdesign gilt:

$$T_{\text{Anwendung}}(n) = \Theta(n^3).$$

Diese Schranke ist unter SETH asymptotisch optimal: Kein Algorithmus kann das zugrundeliegende Subgraph-Problem in $O(n^{3-\varepsilon})$ für $\varepsilon > 0$ lösen [1, 3].

Beweis. Obere Schranke $O(n^3)$:

- FSM-Zustandsminimierung (Satz 4): $T_{\text{FSM}} = O(n^3)$.

- Zustandskodierung (Satz 8): $T_{\text{Kodi}} = O(n^3)$.
- Formale Äquivalenzprüfung (Satz 14): $T_{\text{FEC}} = O(n^3)$.
- Logik-Synthese (Satz 17): $T_{\text{Syn}} = O(n^3)$.
- High-Level-Synthese (Satz 22): $T_{\text{HLS}} = O(n^3)$.
- Statische Zeitanalyse (Satz 25): $T_{\text{STA}} = O(n^3)$.
- ATPG (Satz 32): $T_{\text{ATPG}} = O(n^3)$.

Untere Schranke $\Omega(n^3)$ (unter SETH): In jeder Anwendung ist ein Teilproblem enthalten, das eine LCS-Berechnung auf Sequenzen der Länge $\Theta(n)$ über $\Theta(n)$ Rotationen erfordert. Nach [1, Satz 1] und [3] gilt damit:

$$T_{\text{Anwendung}}(n) \geq \Omega(n^3) \quad \text{unter SETH.}$$

Gesamtergebnis:

$$\Omega(n^3) \leq T_{\text{Anwendung}}(n) \leq O(n^3) \implies T_{\text{Anwendung}}(n) = \Theta(n^3). \quad \square$$

10.1 Numerische Illustration der Laufzeitverhältnisse

Beispiel 7 (Laufzeitvergleich für typische Designgrößen). Betrachte vier typische Probleminstanzen:

Anwendung	n	Naive Per. $O(n!)$	$O(n^3)$	Verhältnis
FSM-Minimierung (8 Zustände)	8	40 320	512	$78,75 \times$
FSM-Minimierung (16 Zustände)	16	$2,09 \times 10^{13}$	4 096	$5,1 \times 10^9 \times$
Zustandskodierung (16 Zust.)	16	$16! = 2,09 \times 10^{13}$	4 096	$5,1 \times 10^9 \times$
Form. Äquivalenzprüfung (64)	64	∞	$2,6 \times 10^5$	∞
Logik-Synthese (100 Gatter)	100	∞	10^6	∞
High-Level-Synthese (200 FUs)	200	∞	8×10^6	∞
Timing-Analyse (1000 Knoten)	1000	∞	10^9	∞
ATPG (500 Knoten)	500	∞	$1,25 \times 10^8$	∞

Der Subgraph Algorithmus macht Designprobleme handhabbar, die mit naiver Enumeration unzugänglich wären. Für $n = 16$ FSM-Zustände ist er über 5 Milliarden Mal schneller als ein naiver Vergleich; für alle Anwendungen mit $n \geq 20$ ist die naive Permutationsaufzählung praktisch unhandhabbar.

11 Zusammenfassung

Diese Arbeit hat sieben fundamentale Anwendungsgebiete des Subgraph Algorithmus aus [1] im digitalen Schaltungsdesign nach Ciletti [2] identifiziert und formal analysiert. Die zentralen Beiträge gegenüber dem Stand der Technik sind: (a) die erstmalige systematische Erfassung aller relevanten Graphmodelle des VLSI-Entwurfsflusses als Instanzen eines einzigen Subgraph-Problems, (b) strenge Laufzeitnachweise für jede Anwendung, und (c) der Nachweis der asymptotischen Optimalität unter SETH für alle sieben Domänen.

Kernresultate:

1. **FSM-Zustandsminimierung (Anwendung I):** Zustandsübergangsgraphen endlicher Automaten werden als Adjazenzmatrizen repräsentiert. Der Subgraph Algorithmus entscheidet Subgraph-Inklusion zweier STGs in $\Theta(n^3)$ und ermöglicht damit automatisierte Äquivalenzprüfung und Zustandsreduktion in Zeit $\Theta(n^3)$ (Satz 4). Für $n = 16$ Zustände ist der Algorithmus $5,1 \times 10^9$ -mal schneller als die naive Permutationsaufzählung.
2. **Zustandskodierung nach FSM-Minimierung (Anwendung II):** Jede Kodierung eines minimierten Automaten erzeugt einen STG, der zu allen anderen Kodierungen isomorph ist (Lemma 1). Der Subgraph Algorithmus erkennt diesen Isomorphismus in $O(m^3)$ (Satz 5), unterstützt Symmetriebrechung in der Kodierungsoptimierung und reduziert den Suchraum von $m!$ auf $m!/|\text{Aut}(G_M)|$ (Satz 7).
3. **Formale Äquivalenzprüfung (Anwendung III):** Ein zweistufiges Verfahren (Subgraph Algorithmus als $O(n^3)$ -Vorfilter + exakter SAT- bzw. Hopcroft-Karp-Test) prüft kombinatorische und sequentielle Äquivalenz korrekt und vollständig (Korollar 2). In der Praxis eliminiert der Vorfilter die große Mehrheit der Paarvergleiche, sodass die Gesamtlaufzeit $O(n^3)$ beträgt (Satz 14).
4. **Logik-Synthese und DAG-Optimierung (Anwendung IV):** Boolesche Funktionsnetze als DAGs unterliegen Subgraph-Matching-Operationen bei Extraktion, Faktorisierung und Technology Mapping. Der Subgraph Algorithmus löst alle diese Schritte in $\Theta(n^3)$ (Sätze 15, 16, 17).
5. **High-Level-Synthese (Anwendung V):** Datenflussgraphen (DFGs) beschreiben algorithmische Berechnungen. Der Subgraph Algorithmus identifiziert isomorphe Funktionseinheiten für Ressourcenwiederverwendung, Pipelining und Replizierung in $\Theta(n^3)$ (Sätze 19, 20, 21, 22).
6. **Statische Zeitanalyse (Anwendung VI):** Timing-DAGs modellieren Signalausbreitung in synchronen Schaltungen. Der Subgraph Algorithmus findet kritische Pfadmuster und unterstützt Setup-Margin-Verifikation in $\Theta(n^3)$ (Sätze 24, 25, 26). Designs mit bis zu $n = 10^6$ Timing-Knoten sind in praktikablen Zeiten analysierbar.
7. **Automatische Testmustergenerierung – ATPG (Anwendung VII):** Stuck-at-Fehler an Schaltungssignalen ändern den Netzlistengraph um genau eine Kante (Lemma 2). Der Subgraph Algorithmus erkennt diese Abweichung durch inkrementelle Signaturaktualisierung für alle $O(n^2)$ Fehler in amortisiert $O(n^3)$ (Satz 32). Zusätzlich beschleunigt er den klassischen D-Algorithmus von $O(n^4)$ auf $O(n^3)$ (Korollar 3) und ermöglicht greedy-optimale Testmustergenerierung mit $(1 - 1/e)$ -Approximationsgarantie (Satz 30).

Einheitliches Graphmodell: Der zentrale theoretische Beitrag ist Satz 33: Alle sieben Designabstraktionen (STG, kodierter STG, Miter, Boolesches Netz, DFG, Timing-DAG, Fehlergraph) lassen sich einheitlich als gewichtete gerichtete Graphen mit Adjazenzmatrix darstellen. Der Subgraph Algorithmus ist daher ein universeller $\Theta(n^3)$ -Kern für den gesamten Entwurfsfluss.

Optimalität: Unter der Strong Exponential Time Hypothesis (SETH) ist $\Theta(n^3)$ asymptotisch optimal für alle sieben Anwendungen (Satz 34). Dies folgt aus der SETH-bedingten unteren Schranke für LCS [3] und der notwendigen Prüfung aller n zyklischen Rotationen [1]. Unbedingt (ohne Hypothese) gilt $\Omega(n^2)$ nach einem Kommunikationskomplexitätsargument.

12 Ausblick

Die in dieser Arbeit erarbeiteten Ergebnisse öffnen eine Reihe substanzieller Forschungsrichtungen. Im Folgenden werden diese ausführlich diskutiert und, wo möglich, mit ersten formalen Abschätzungen unterlegt.

12.1 Parallelisierung auf Mehrkernprozessoren und GPUs

Der Subgraph Algorithmus besitzt eine natürliche Parallelstruktur: Die n zyklischen Rotationen in Schritt 3 sind vollständig unabhängig voneinander und können ohne Synchronisation parallel ausgeführt werden. Für eine p -Kern-Maschine ergibt sich sofort:

$$T_{\text{Subgraph,par}}(n, p) = O\left(\frac{n^3}{p}\right) + O(n^2),$$

wobei der verbleibende Term $O(n^2)$ die einmalige Signaturberechnung (inhärent sequentiell) widerspiegelt. Für $p = \Theta(n)$ gilt damit:

$$T_{\text{Subgraph,par}}(n, n) = O(n^2),$$

was der LCS-Schranke eines einzelnen Vergleichs entspricht – eine asymptotisch optimale Parallelisierung unter PRAM [19].

Auf modernen Grafikprozessoren (GPUs) stehen $p = O(10^4)$ bis $p = O(10^5)$ SIMD-Ausführungseinheiten zur Verfügung. Für Timing-DAGs mit $n = 10^6$ Knoten:

$$T_{\text{GPU}} = O\left(\frac{(10^6)^3}{10^5}\right) = O(10^{13}) \quad \text{Operationen.}$$

Dies ist zwar im absoluten Sinne groß, jedoch durch GPU-Datendurchsatz ($\approx 10^{12}$ FLOPS) in unter 10 Sekunden ausführbar – verglichen mit Stunden auf einem Einzel-CPU-System.

Das zentrale offene Problem ist die *LCS-Berechnung auf GPU*: Da DP-Algorithmen Datenabhängigkeiten besitzen, ist die direkte Parallelisierung nicht trivial. Anti-diagonale Aufteilung der DP-Tabelle erlaubt jedoch $O(n)$ parallele Schritte, von denen jeder $O(n)$ Elemente parallel berechnet – ein Gesamtaufwand von $O(n)$ Zeitschritten mit $O(n)$ Parallelität, also $O(n^2)$ Arbeit je Rotation [20].

Offene Frage: Lässt sich die GPU-Implementierung des Subgraph Algorithmus so gestalten, dass der Speichertransfer (CPU $\leftrightarrow$ GPU) die Ausführungszeit nicht dominiert? Die Speicherbandbreite heutiger High-End-GPUs ($\approx 1,5$ TB/s bei NVIDIA H100) erlaubt das Einlesen einer $n \times n$ -Matrix mit $n = 10^4$ in unter 1 ms – was den Engpass auf die DP-Berechnung verlagert, nicht auf den Datentransfer.

12.2 Integration in industrielle EDA-Werkzeuge

Aktuelle Electronic Design Automation (EDA)-Werkzeuge – insbesondere Synopsys Design Compiler [27], Cadence Genus und Mentor Precision RTL – implementieren Technology Mapping und Logikoptimierung mit proprietären Heuristiken, die typischerweise eine Kombination aus Greedy-Algorithmen und lokaler Suche verwenden. Die Laufzeit dieser Verfahren ist empirisch $O(n^{3.5})$ bis $O(n^4)$ für schlimmste Eingaben [15].

Die Integration des Subgraph Algorithmus als universeller $\Theta(n^3)$ -Kern böte folgende Vorteile:

12.2.1 Vereinfachung der Werkzeug-Architektur

Anstatt separate Algorithmen für Extraktion, Faktorisierung, Technology Mapping und FSM-Äquivalenzprüfung zu implementieren, könnte ein einziges hochoptimiertes Subgraph-Modul alle vier Aufgaben übernehmen. Die Codebasis würde sich schätzungsweise um den Faktor 3–5 reduzieren, da Extraktion, Faktorisierung und Technology Mapping alle auf dieselbe Grundoperation (subsequenzielle Signaturprüfung) zurückgeführt werden.

12.2.2 Wiederholbarkeit und Verifikation

Proprietäre EDA-Werkzeuge liefern für identische Eingaben manchmal unterschiedliche Ergebnisse (Nicht-Determinismus durch zeitabhängige Heuristiken). Da der Subgraph Algorithmus deterministisch und vollständig definiert ist, würde seine Integration die Reproduzierbarkeit der Synthese-Ergebnisse garantieren – ein zentrales Qualitätsmerkmal in der funktionalen Sicherheit (ISO 26262, DO-178C).

12.2.3 Formales Zertifizierungspotenzial

In sicherheitskritischen Designs (Automobil, Luftfahrt, Medizin) ist ein mathematisch bewiesenes Laufzeitbudget ($\Theta(n^3)$) wertvoller als eine empirisch beobachtete Laufzeit. Der Subgraph Algorithmus könnte als *zertifizierbarer Kern* in Synthesewerkzeugen dienen, die für Safety-Assessments eingesetzt werden.

Konkrete nächste Schritte:

1. Implementierung eines Open-Source EDA-Plugins (z. B. als Yosys [17] Pass) in C++.
2. Benchmarking auf den ISCAS-85 und ISCAS-89 Testkreisen [18] mit Vergleich gegen MISII und ABC [16].
3. Messung der Wallclock-Zeit und Silizium-Qualität (Fläche, kritischer Pfad) der synthetisierten Schaltungen.

12.3 Erweiterung auf gewichtete und annotierte Graphen

Der Subgraph Algorithmus wurde in [1] für ungewichtete Graphen definiert. Für die digitale Schaltungsanalyse wären jedoch gewichtete Varianten hochrelevant:

12.3.1 Timing-DAGs mit Verzögerungsgewichten

Im Timing-DAG trägt jede Kante (u, v) eine Ausbreitungsverzögerung $d(u, v) \in \mathbb{R}_{>0}$. Ein Subgraph $G_P \subseteq G_T$ soll nicht nur strukturell, sondern auch *zeitlich* passen: Die summierten Pfadverzögerungen im Teilgraphen sollen innerhalb eines vorgegebenen Toleranzfensters $[\tau_{\min}, \tau_{\max}]$ liegen.

Definition 27 (Gewichteter Subgraph-Match). Ein Teilgraph $G_P = (V_P, E_P, d_P)$ ist ein *gewichteter Subgraph* von $G_T = (V_T, E_T, d_T)$, wenn:

1. $G_P \subseteq G_T$ im ungewichteten Sinne (strukturelle Einbettung), und
2. für alle Pfade $\pi \in G_P$ gilt: $\tau_{\min} \leq \sum_{(u,v) \in \pi} d_T(\varphi(u), \varphi(v)) \leq \tau_{\max}$.

Die Erweiterung des Subgraph Algorithmus auf diesen Fall erfordert eine modifizierte Signatur, die Kantengewichte einbezieht. Eine natürliche Erweiterung ist:

$$\sigma_j^{(w)} = \sum_{i=0}^{n-1} A_{ij} \cdot \lfloor d(i, j) / \delta \rfloor \cdot 2^i + j \cdot 2^{n \cdot W},$$

wobei δ eine Quantisierungsaufösung und $W = \lceil \log_2 \tau_{\max} / \delta \rceil$ die Anzahl der Bits pro Gewicht ist. Die Laufzeit der erweiterten Version steigt auf $O(n^3 \cdot W)$ – polynomiell in Eingabegröße und Gewichtspräzision.

12.3.2 DFGs mit Ressourcen-Annotations

In DFGs können Knoten mit Ressourcentypen annotiert sein (z.B. “Multiplizierer”, “Addierer”). Ein Subgraph-Match soll dann nicht nur struktur-, sondern auch ressourcentyperhaltend sein. Die Signatur kann um einen Typterm erweitert werden:

$$\sigma_j^{(\text{typ})} = \sigma_j + \text{hash}(\text{typ}(v_j)) \cdot 2^{2n}.$$

Da diese Erweiterung die Injektivität der Signatur-Funktion beibehält (disjunkte Intervalle), sind Korrektheit und Laufzeit $\Theta(n^3)$ unverändert.

12.4 Approximative Varianten für inkrementelle Re-Synthese

In inkrementellen Synthese-Workflows (z. B. nach einem lokalen RTL-Bugfix) muss nur ein kleiner Teil der Schaltung neu synthetisiert werden. Hier ist eine exakte $\Theta(n^3)$ -Lösung oft überdimensioniert.

12.4.1 (ε, δ) -Approximation

Wir definieren eine $(1 - \varepsilon)$ -Approximation des Subgraph-Problems: Statt zu prüfen, ob $G \subseteq G'$ exakt, prüfen wir, ob ein Teilgraph $G'' \subseteq G'$ mit $|V(G'')| \geq (1 - \varepsilon)|V(G)|$ und $|E(G'')| \geq (1 - \varepsilon)|E(G)|$ existiert.

Satz 35 (Approximative Subgraph-Prüfung). Eine $(1 - \varepsilon)$ -Approximation des zyklischen Subgraph-Problems lässt sich in Zeit $O(n^2 \cdot \lceil 1/\varepsilon \rceil)$ berechnen, indem nur $\lceil n \cdot \varepsilon \rceil$ zufällig gewählte Rotationen geprüft werden.

Beweis. Sei $k = \lceil n \cdot \varepsilon \rceil$ die Anzahl der geprüften Rotationen. Nach dem probabilistischen Argument: Falls ein exakter Match bei Rotation r^* existiert, ist die Wahrscheinlichkeit, r^* nicht zu treffen, höchstens:

$$\Pr[\text{Keine Auswahl von } r^*] = \left(1 - \frac{1}{n}\right)^k \leq e^{-k/n} = e^{-\varepsilon}.$$

Für $\varepsilon = 0.1$: Trefferwahrscheinlichkeit $\geq 1 - e^{-0.1} \approx 9.5\%$ pro Stichprobe. Mit $k = \lceil 10 \ln(1/\delta) \rceil$ Rotationen (Boosting) erhält man eine Fehlerwahrscheinlichkeit von $\leq \delta$. Laufzeit: $k \cdot O(n^2) = O(n^2/\varepsilon)$. $\square$

Praktische Relevanz: Für einen inkrementellen RTL-Patch, der $\varepsilon = 5\%$ der Schaltungsknoten betrifft, reduziert sich der Verifikationsaufwand von $O(n^3)$ auf $O(20 \cdot n^2)$ – ein Faktor $n/20$ Beschleunigung, also für $n = 1000$ ein 50-facher Speed-Up.

12.5 Bedingte Untere Schranken und Komplexitätstheoretische Perspektiven

12.5.1 Jenseits von SETH: Fine-Grained Complexity

Die in [1] verwendete SETH-Schranke ist nur eine von mehreren Komplexitätshypothesen der *Fine-Grained Complexity* [23]. Weitere relevante Hypothesen:

- **OV-Hypothese** (Orthogonal Vectors): Zwei Mengen von n Vektoren in $\{0, 1\}^d$ können nicht in $O(n^{2-\varepsilon} \cdot d)$ auf orthogonale Vektorpaare geprüft werden. Da Subgraph-Signaturvergleich OV-reduzierbar ist, folgt: $T_{\text{Subgraph}} \geq \Omega(n^{2-\varepsilon})$ auch unter OV-Hypothese (unabhängig von SETH).
- **3SUM-Hypothese**: Drei Mengen von n Zahlen können nicht in $O(n^{2-\varepsilon})$ auf eine Dreijahressumme ($a + b + c = 0$) geprüft werden. Das Subgraph-Problem lässt sich in bestimmten Varianten auf 3SUM reduzieren, was eine $\Omega(n^{4/3})$ -Schranke impliziert – schwächer als SETH, aber ohne Hypothese über SAT.

Satz 36 (Bedingte Schranke unter OV-Hypothese). Unter der OV-Hypothese gilt für das zyklische Subgraph-Problem:

$$T_{\text{Subgraph}}(n) \geq \Omega(n^{2-\varepsilon}) \quad \text{für alle } \varepsilon > 0.$$

Beweisidee. Das Signatur-Vergleichsproblem des Subgraph Algorithmus ist eine Instanz des Vektorähnlichkeitsproblems: Gegeben Sequenzen Σ^A und Σ^B über einem Integer-Alphabet, prüfe ob eine zyklische Verschiebung von Σ^B einen langen gemeinsamen Teilstring mit Σ^A besitzt. Dies entspricht dem *Offline Longest Common Subsequence*-Problem auf n Vektoren der Dimension n , das unter OV nicht in $O(n^{2-\varepsilon} \cdot n)$ lösbar ist [24]. Damit gilt $T_{\text{Subgraph}} \geq \Omega(n^{2-\varepsilon})$. $\square$

12.5.2 Unbedingte untere Schranke: Kommunikationskomplexität

Für eine unbedingte (hypothesenfreie) $\Omega(n^2)$ -Schranke kann ein Kommunikationskomplexitäts-Argument verwendet werden [25]: Alice hält Matrix A , Bob hält Matrix B . Um $G \subseteq G'$ zu entscheiden, muss mindestens $\Omega(n^2)$ Information ausgetauscht werden (da die Eingabe $2n^2$ Bits umfasst und die Ausgabe von allen Bits abhängen kann). Dies beweist ohne jede Hypothese:

$$T_{\text{Subgraph}}(n) \geq \Omega(n^2) \quad (\text{unbedingt}).$$

12.6 Erweiterung auf gerichtete azyklische Multigraphen und Hypergraphen

In modernen VLSI-Designs treten Strukturen auf, die über einfache Digraphen hinausgehen:

12.6.1 Multigraphen für mehrfach verbundene Netze

In einem Routing-Graphen können zwei Zellen durch mehrere Verdrahtungsoptionen verbunden sein (z. B. Metall-1, Metall-2, Metall-3 in einem Dreischicht-Prozess). Ein *Multigraph* erlaubt Mehrfachkanten zwischen denselben Knoten:

$$E \subseteq V \times V \times L, \quad L = \{\text{met1}, \text{met2}, \text{met3}\}.$$

Die Adjazenzmatrix wird zur Adjazenzmatrix-Familie $\{A^{(l)}\}_{l \in L}$, eine für jede Schicht. Der Subgraph Algorithmus lässt sich direkt auf Multigraphen erweitern, indem die Signatur um Schicht-Terme ergänzt wird:

$$\sigma_j^{(l)} = \sum_{i=0}^{n-1} A_{ij}^{(l)} \cdot 2^i + j \cdot 2^n + \text{hash}(l) \cdot 2^{2n}.$$

Für $|L|$ Schichten steigt die Laufzeit auf $O(|L| \cdot n^3)$ – linear in der Anzahl der Schichten.

12.6.2 Hypergraphen für Multi-Fanout-Netze

In Logiknetzen kann ein Gate-Ausgang an beliebig viele Gate-Eingänge verbunden sein (Fanout > 2). Dies wird durch Hyperkanten modelliert. Ein *gerichteter Hypergraph* $H = (V, \mathcal{E})$ hat Hyperkanten $e \in \mathcal{E}$ mit $e \subseteq V \times 2^V$. Die Adjazenzmatrix wird zur *Inzidenzmatrix* $B \in \{0, 1\}^{n \times m}$ (Knoten $\times$ Hyperkanten).

Die Erweiterung des Signaturkonzepts auf Inzidenzmatrizen ist nicht-trivial, da die LCS-basierte Rotationsprüfung auf rechteckigen Matrizen der Form $n \times m$ (mit $m \neq n$) operate. Erste Vorstudien legen nahe, dass für Hypergraphen mit maximalem Fanout f eine Laufzeit von $O(f \cdot n^3)$ erreichbar ist.

12.7 Automatische Verifikation digitaler Systeme

12.7.1 Hardware-Model-Checking mit Subgraph-Unterstützung

Model Checking [26] prüft, ob ein endlicher Automat M eine temporallogische Spezifikation φ erfüllt. Ein zentrales Teilproblem ist die *Bisimulation*: Zwei Automaten M_1 und M_2 sind bisimulationsäquivalent genau dann, wenn jeder Übergang von M_1 durch einen entsprechenden Übergang von M_2 simuliert werden kann und umgekehrt.

Bisimulation ist stärker als Subgraph-Inklusion (sie erfordert symmetrisches Matching), aber der Subgraph Algorithmus kann als effizienter Vorfilter in einem zweistufigen Verfahren dienen:

1. **Vorstufe:** Prüfe $G_{M_1} \subseteq G_{M_2}$ und $G_{M_2} \subseteq G_{M_1}$ mit dem Subgraph Algorithmus in $O(n^3)$. Falls eines scheitert, ist Bisimulation ausgeschlossen.
2. **Nachstufe:** Führe exaktes Bisimulations-Checking durch (z. B. Hopcroft-Karp [9] in $O(n \log n)$).

Dieses zweistufige Vorgehen eliminiert in der Praxis den teuren zweiten Schritt für alle Nicht-Bisimulations-Paare und reduziert die durchschnittliche Laufzeit signifikant.

12.7.2 Property-Checking in Verilog-Designs

Moderne Verilog-Syntheseflüsse beinhalten Assertion-Checking (SVA, SystemVerilog Assertions). Eine Assertion $\forall t : P(s_t)$ über einen Signalpfad s_t des Designs lässt sich als *Mustererkennung im STG* auffassen: Der STG des Designs darf den Teilgraphen $G_{\neg P}$ (der die Verletzungszustände der negierten Eigenschaft darstellt) nicht enthalten.

Satz 37 (Assertion-Prüfung via Subgraph). Gegeben ein Verilog-Design mit STG G_M (n Zustände) und eine SVA-Assertion, die durch einen Constraintgraphen $G_{\neg P}$ ($m \leq n$ Knoten) repräsentiert wird. Die Assertion wird verletzt, genau wenn $G_{\neg P} \subseteq G_M$. Dieser Test hat Laufzeit $O(n^3)$.

Beweis. Die Korrektheit folgt direkt aus der Definition des Subgraph Algorithmus [1]: Falls $G_{\neg P} \subseteq G_M$, gibt es eine Zustandsfolge im Design, die die Assertion verletzt. Die Laufzeit $O(n^3)$ folgt aus Satz 4. $\square$

12.8 Maschinelles Lernen und neuronale Netzarchitekturen

12.8.1 Graph Neural Networks für Subgraph-Erkennung

Neuronale Graphennetzwerke (Graph Neural Networks, GNNs) [28] können auf Graphen trainiert werden, um Subgraph-Inklusion approximativ zu entscheiden. Dies ist relevant, wenn eine hohe Durchsatzrate wichtiger ist als exakte Korrektheit (z. B. beim schnellen Screening von Millionen von Designvarianten).

Ein GNN der Tiefe L mit versteckter Dimension d hat Inferenzlaufzeit $O(L \cdot (n \cdot d + |E| \cdot d^2))$ – für $d = O(1)$ und $L = O(1)$ linear in n . Der Subgraph Algorithmus hingegen ist exakt und garantiert korrekt.

Hybridansatz: Ein zweistufiges System mit GNN-Vorfilter ($O(n)$) und nachgelagertem exaktem Subgraph Algorithmus ($O(n^3)$) nur für vielversprechende Kandidaten könnte lineare durchschnittliche Laufzeit bei garantierter Korrektheit erreichen.

12.8.2 Lernbasierte Signaturverbesserung

Die Signatur-Funktion $\sigma_j = \sum_i A_{ij} \cdot 2^i + j \cdot 2^n$ ist in [1] analytisch hergeleitet. Es wäre interessant zu untersuchen, ob eine *gelernte Einbettung* $\phi : \{0, 1\}^n \rightarrow \mathbb{R}^k$ mit $k \ll n$ die Rotationsprüfung beschleunigen kann, falls Kollisionen (falsche LCS-Treffer die keine echten Subgraphen sind) toleriert werden.

12.9 Anwendungen in neuen Halbleiterprozessen

12.9.1 3D-ICs und Die-Stacking

In dreidimensionalen integrierten Schaltkreisen (3D-ICs) werden mehrere Dies übereinander gestapelt und mit Through-Silicon Vias (TSVs) verbunden. Der Verbindungsgraph ist kein planarer Graph mehr, sondern ein Mehrebenengraph $G_{3D} = (V, E_{\text{intra}} \cup E_{\text{inter}})$, wobei E_{intra} Verbindungen innerhalb einer Schicht und E_{inter} Die-übergreifende TSV-Verbindungen bezeichnen. Der Subgraph Algorithmus kann auf jede Schicht separat angewendet werden ($O(n_i^3)$ je Schicht i) und die TSV-Struktur als Kantenannotation einbeziehen ($O(k \cdot n^3)$ für k Schichten mit je n Knoten).

12.9.2 Quantenschaltkreise

Quantenschaltkreise werden als gerichtete Graphen über einem komplexen Alphabet (unitäre Gatter) repräsentiert. Obwohl die Adjazenzmatrix über $\mathbb{C}$ definiert ist, kann der Subgraph Algorithmus mit komplexen Signaturen angepasst werden:

$$\sigma_j^{\mathbb{C}} = \sum_{i=0}^{n-1} |U_{ij}|^2 \cdot p_i + j \cdot 2^n,$$

wobei p_i Primzahlen sind (zur Eindeutigkeit). Gleichheit zweier Signaturen ist dann ein notwendiges (aber nicht hinreichendes) Kriterium für unitäre Äquivalenz zweier Quantengatter-Anordnungen. Der Algorithmus dient als effizienter Vorfilter für Quantenschaltungsoptimierung [29].

12.10 Weiterführende Perspektiven für Zustandskodierung

Die Ergebnisse aus Kapitel 3 eröffnen mehrere konkrete Forschungsrichtungen:

12.10.1 Lernbasierte Kodierungsoptimierung

Das Problem der optimalen Zustandskodierung ist NP-hart (Satz 7). Der Subgraph Algorithmus reduziert den Suchraum von $m!$ auf $m!/|\text{Aut}(G_M)|$. Eine natürliche Erweiterung ist ein Machine-Learning-Ansatz: Trainiere ein neuronales Netzwerk, das aus den STG-Eigenschaften direkt eine hochqualitative Kodierung κ vorhersagt, ohne den gesamten Suchraum zu durchsuchen. Der Subgraph Algorithmus dient dann als effizienter Validator ($O(m^3)$) im Trainingsloop.

Offene Frage: Gibt es eine Klasse von STGs (z. B. reguläre Graphen), für die das Zustandskodierungsproblem polynomial lösbar ist?

12.10.2 Mehrziel-Kodierungsoptimierung

In der Praxis sind Gatterkomplexität, Taktfrequenz und Verlustleistung konkurrierende Ziele. Wir definieren:

$$\kappa^* = \arg \min_{\kappa} (\alpha \cdot W(\kappa) + \beta \cdot P(\kappa) + \gamma \cdot D(\kappa)),$$

wobei W das Kodierungsgewicht (Satz 8), P die Verlustleistung (proportional zur Schaltaktivität) und D die kritische Pfadverzögerung bezeichnen. Der Subgraph Algorithmus kann alle drei Terme durch Matchingoperationen auf dem kodierten STG berechnen. Die Pareto-Optimierung dieses Mehrzielproblems bleibt offen.

12.11 Perspektiven für formale Äquivalenzprüfung

12.11.1 Inkrementelle Äquivalenzprüfung bei RTL-Änderungen

In modernen Entwurfsumgebungen werden RTL-Beschreibungen kontinuierlich geändert (z. B. agile Hardware-Entwicklung). Nach jeder Änderung muss die Äquivalenz zur Vorversion geprüft werden. Der Subgraph Algorithmus eignet sich besonders gut für *inkrementelle* FEC:

Satz 38 (Inkrementelle Äquivalenzprüfung). Wenn eine RTL-Änderung lokal k Knoten des Netzlistengraphen betrifft ($k \ll n$), kann die inkrementelle Subgraph-basierte Äquivalenzprüfung in Zeit $O(k \cdot n^2)$ durchgeführt werden, anstatt in $O(n^3)$ für den vollständigen Neutest.

Beweis. Nur die Signaturen der k geänderten Knoten müssen aktualisiert werden: $O(k \cdot n)$. Die LCS-Berechnung zwischen den k neuen und allen n alten Signaturen kostet $O(k \cdot n^2)$. Gesamtlaufzeit: $O(k \cdot n^2) \ll O(n^3)$ für $k \ll n$. $\square$

12.11.2 Integration in ISO-26262-konforme Entwurfsflüsse

Die formale Äquivalenzprüfung ist in funktional-sicherheitskritischen Entwurfsflüssen nach ISO 26262 (Automotive) und DO-178C (Avionics) verpflichtend. Der Subgraph Algorithmus bietet gegenüber proprietären Werkzeugen den Vorteil des *beweisbaren Laufzeitbudgets*: $T_{\text{FEC}} = O(n^3)$ ist formal nachgewiesen, nicht nur empirisch beobachtet. Dies ist ein

entscheidender Vorteil für das Safety Case Assessment, da Werkzeugqualifizierung (TQL-1 nach DO-330) einen dokumentierten, verifizierten Algorithmus erfordert.

Konkrete nächste Schritte:

1. Implementierung des zweistufigen FEC-Algorithmus als quelloffener Yosys-Pass [17] in C++.
2. Benchmarking auf dem ITC-99-Testsatz (sequentielle Benchmark-Schaltkreise).
3. Formale Verifikation der Implementierung mit einem Proof-Assistent (Coq, Lean).

12.12 Weiterführende Perspektiven für ATPG

12.12.1 ATPG für Mehrfachfehler

Das klassische Stuck-at-Fehlermodell betrachtet Einzelfehler. In modernen Nanometer-Prozessen treten jedoch gehäuft Mehrfachfehler auf (Multiple Simultaneous Faults, MSF). Ein MSF ändert mehrere Kanten im Netzlistengraph gleichzeitig.

Definition 28 (*k*-facher Stuck-at-Fehler). Ein *k*-facher Stuck-at-Fehler $F_k = \{f_1, \dots, f_k\}$ erzeugt eine Adjazenzmatrixänderung:

$$A_{F_k} = A_N + \sum_{l=1}^k \varepsilon_l \cdot \Delta_{f_l}, \quad \varepsilon_l \in \{-1, +1\},$$

wobei $\|\Delta_{f_l}\|_1 = 1$ für jeden Einzelfehler.

Satz 39 (Subgraph Algorithmus für *k*-fache Fehler). Ein *k*-facher Stuck-at-Fehler F_k ist strukturell detektierbar genau dann, wenn $G_{F_k} \not\subseteq G_N$. Der Subgraph Algorithmus entscheidet dies in $O(n^3)$, unabhängig von *k*.

Beweis. Für beliebiges *k* gilt: A_{F_k} unterscheidet sich von A_N in höchstens *k* Einträgen. Die Signatur $\Sigma^{A_{F_k}}$ unterscheidet sich von Σ^{A_N} in höchstens *k* Spalten. Die modifizierte Signatursequenz ist in $O(k \cdot n)$ inkrementell berechnet. Der LCS-Test bleibt $O(n^2)$ je Rotation und $O(n^3)$ gesamt. $\square$

12.12.2 Delay-Fehlermodell und Path-Delay-Testing

Das Stuck-at-Modell abstrahiert strukturelle Fehler; reale Fertigungsfehler äußern sich oft als erhöhte Signalverzögerungen (Path Delay Faults). Ein Path-Delay-Fehler f_{del} erhöht das Kantengewicht $d(e)$ im Timing-DAG um $\Delta d > 0$.

Satz 40 (Integriertes ATPG-STA via Subgraph Algorithmus). Das kombinierte Structural / Timing-ATPG (Stuck-at + Path-Delay) kann als Subgraph-Matching auf dem gewichteten Timing-DAG G_T formuliert werden. Der Subgraph Algorithmus mit gewichteter Signatur $\sigma_j^{(w)} = \sum_i A_{ij} \cdot \lfloor d(i, j) / \delta \rfloor \cdot 2^i$ (analog zu Definition 27 im Ausblick der Originalarbeit) löst das Problem in $O(n^3 \cdot W)$ mit Quantisierungsbreite *W*.

Beweis. Ein Path-Delay-Fehler ändert ein Kantengewicht $d(e^*) \rightarrow d(e^*) + \Delta d$. In der quantisierten Signatur ändert sich der entsprechende Term: $\lfloor (d(e^*) + \Delta d) / \delta \rfloor \neq \lfloor d(e^*) / \delta \rfloor$ falls $\Delta d \geq \delta$. Damit ist $\sigma_j^{(w, \text{fault})} \neq \sigma_j^{(w)}$ – der Subgraph Algorithmus detektiert den Fehler. Laufzeit: Signaturberechnung mit *W* Bits pro Gewichtseintrag kostet $O(n^2 W)$; LCS-Test kostet $O(n^2)$ je Rotation, Gesamtlaufzeit $O(n^3 \cdot W)$. $\square$

12.12.3 Diagnosefähigkeit: Fehlerlokalisierung statt nur Detektion

Über die Detektion hinaus ist die *Fehlerlokalisierung* (Fault Diagnosis) wichtig: Welcher konkrete Fehler $f \in \mathcal{F}$ ist in einem defekten Chip aufgetreten? Die Diagnose vergleicht gemessene Testantworten mit simulierten Fehlerantworten.

Satz 41 (Fehlerlokalisierung via Subgraph Algorithmus). Für einen defekten Chip mit beobachteter Ausgabesequenz $\mathbf{y}^{\text{obs}}$ kann die Diagnoseliste $\mathcal{D} = \{f \in \mathcal{F} \mid N_f \text{ erklärt } \mathbf{y}^{\text{obs}}\}$ durch Subgraph-Matching aller Fehlergraphen G_f gegen den “Beobachtungsgraph” G_{obs} (der aus $\mathbf{y}^{\text{obs}}$ rekonstruiert wird) in Zeit $O(|\mathcal{F}| \cdot n^3/n) = O(n^4)$ ermittelt werden; mit Signatur-Caching amortisiert auf $O(n^3)$.

Beweis. Der Beobachtungsgraph G_{obs} wird aus der gemessenen Ausgabe rekonstruiert (Modell-basierte Diagnose): $O(n)$. Für jeden Fehler $f \in \mathcal{F}$ ($|\mathcal{F}| = O(n^2)$):

- Prüfe inkrementell $G_f \subseteq G_{\text{obs}}$: $O(n^2)$ mit gecachten Signaturen.

Gesamt: $O(n^2) \cdot O(n^2)/n = O(n^3)$ amortisiert (Signatur-Caching wie in Satz 28). □

12.13 Formale Offene Probleme

Wir schließen den Ausblick mit einer erweiterten Liste formaler offener Fragen, von denen jede eine eigenständige Forschungsarbeit rechtfertigt:

1. **Unbedingte $\Omega(n^3)$ -Schranke:** Kann das zyklische Subgraph-Problem ohne Rückgriff auf Komplexitätshypothesen (SETH, OV) in $\Omega(n^3)$ bewiesen werden? Dies ist ein offenes Problem der Fine-Grained Complexity.
2. **Sub-kubische Algorithmen unter Einschränkungen:** Für welche Graphklassen (z. B. planare Graphen, Graphen mit beschränkter Baumweite, DAGs mit kleinem Fanin) lässt sich das Subgraph-Problem in $O(n^{2.5})$ oder besser lösen?
3. **Online-Variante:** Gegeben ein wachsender STG (neue Zustände kommen inkrementell hinzu). Kann die Subgraph-Prüfung in $O(n^2)$ amortisierter Zeit pro Einfügeoperation aufrechterhalten werden (analog zu dynamischen Graphalgorithmen)?
4. **Parallelität und Raumkomplexität:** Kann der Subgraph Algorithmus in der PRAM-Klasse $\mathcal{NC}^2$ (polylogarithmische Zeit, polynomiell viele Prozessoren) oder $\mathcal{NC}^1$ realisiert werden?
5. **Randomisierte Beschleunigung:** Gibt es einen Las-Vegas-Algorithmus, der das zyklische Subgraph-Problem im Erwartungswert in $O(n^{2.5})$ löst?
6. **Erweiterung auf Graphen verschiedener Größe mit $n_A \neq n_B$:** Der aktuelle Algorithmus ist auf $n_A \leq n_B$ ausgelegt. Die allgemeine Version (verschiedene Knotenmengen, Einbettung durch beliebige – nicht nur zyklische – Permutation) ist NP-vollständig [5]. Die Grenze zwischen polynomiellen und NP-schweren Varianten ist noch nicht vollständig charakterisiert.
7. **Optimale Zustandskodierung in Polynomialzeit:** Gibt es eine Klasse von FSM-Graphen, für die das optimale Kodierungsproblem (minimales Kodierungsgewicht $W(\kappa)$) in Polynomialzeit lösbar ist? Erste Kandidaten sind reguläre Graphen und Graphen mit beschränkter Treewidth.

8. **Vollständige sequentielle Äquivalenzprüfung ohne SAT-Backend:** Kann die sequentielle Äquivalenzprüfung (Bisimulation) rein durch den Subgraph Algorithmus entschieden werden, ohne Rückgriff auf den Hopcroft-Karp-Algorithmus? Hierfür müsste die Bisimulationsrelation direkt aus der Signaturstruktur ablesbar sein.
9. **ATPG für k -fache Fehler mit minimaler Testmenge:** Das Minimum-Set-Cover-Problem für k -fache Stuck-at-Fehler ist für $k \geq 2$ noch schwerer als für $k = 1$. Gibt es approximationserhaltende Reduktionen, die zeigen, dass die $(1 - 1/e)$ -Approximation aus Satz 30 auch für $k \geq 2$ gilt?
10. **Integriertes Synthesis-Verification Framework:** Kann ein einheitliches Werkzeug entwickelt werden, das Logik-Synthese (Anwendung IV), formale Äquivalenzprüfung (Anwendung III) und ATPG (Anwendung VII) in einem gemeinsamen Subgraph Algorithmus-Kern verbindet – mit einer Gesamtlaufzeit $O(n^3)$ für den kompletten Synthesis-and-Verify-Flow?

Literatur

- [1] S. Epp, *Der Subgraph Algorithmus*, Verfügbar unter <https://gitlab.com/epp-group/subgraph>
- [2] M. D. Ciletti, *Advanced Digital Design with the Verilog HDL*, 2. Aufl., Prentice Hall, 2011.
- [3] A. Abboud, A. Backurs, V. V. Williams, *Tight Hardness Results for LCS and Other Sequence Similarity Measures*, in Proc. IEEE FOCS, 2015, S. 59–78.
- [4] R. K. Brayton, R. Rudell, A. Sangiovanni-Vincentelli, A. R. Wang, *MIS: A Multiple-Level Logic Optimization System*, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst., Bd. 6, Nr. 6, S. 1062–1081, 1987.
- [5] S. A. Cook, *The Complexity of Theorem Proving Procedures*, in Proc. ACM STOC, 1971, S. 151–158.
- [6] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, 4. Aufl., MIT Press, 2022.
- [7] S. Devadas, A. Ghosh, K. Keutzer, *Logic Synthesis*, McGraw-Hill, 1994.
- [8] K. L. McMillan, *Symbolic Model Checking*, Kluwer Academic Publishers, 1993.
- [9] J. Hopcroft, R. Karp, *A Linear Algorithm for Testing Equivalence of Finite Automata*, Technical Report 114, Cornell University, 1971.
- [10] A. B. Kahng, J. Lienig, I. L. Markov, J. Hu, *VLSI Physical Design: From Graph Partitioning to Timing Closure*, 2. Aufl., Springer, 2022.
- [11] M. R. Garey, D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*, W. H. Freeman, 1979.
- [12] J. R. Ullmann, *An Algorithm for Subgraph Isomorphism*, J. ACM, Bd. 23, Nr. 1, S. 31–42, 1976.

- [13] L. P. Cordella, P. Foggia, C. Sansone, M. Vento, *A (Sub)Graph Isomorphism Algorithm for Matching Large Graphs*, IEEE Trans. Pattern Anal. Mach. Intell., Bd. 26, Nr. 10, S. 1367–1372, 2004.
- [14] L. G. Valiant, *Universality Considerations in VLSI Circuits*, IEEE Trans. Comput., Bd. 30, Nr. 2, S. 135–140, 1981.
- [15] E. M. Sentovich, K. J. Singh, L. Lavagno, C. Moon, R. Murgai, A. Saldanha, H. Savoj, P. R. Stephan, R. K. Brayton, A. Sangiovanni-Vincentelli, *SIS: A System for Sequential Circuit Synthesis*, Technical Report UCB/ERL M92/41, UC Berkeley, 1992.
- [16] R. Brayton, A. Mishchenko, *ABC: An Academic Industrial-Strength Verification Tool*, in Proc. CAV, Lecture Notes in Computer Science, Bd. 6174, S. 24–40, 2010.
- [17] C. Wolf, *Yosys Open SYnthesis Suite*, Proc. Austrochip, 2013. <https://yosyshq.net>
- [18] F. Brglez, H. Fujiwara, *A Neutral Netlist of 10 Combinational Benchmark Circuits and a Target Translator in Fortran*, in Proc. IEEE ISCAS, S. 663–698, 1985.
- [19] J. Jájá, *An Introduction to Parallel Algorithms*, Addison-Wesley, 1992.
- [20] L. Allison, T. I. Dix, *A Bit-String Longest-Common-Subsequence Algorithm*, Inform. Process. Lett., Bd. 23, Nr. 6, S. 305–310, 1986.
- [21] T. F. Smith, M. S. Waterman, *Identification of Common Molecular Subsequences*, J. Molecular Biology, Bd. 147, Nr. 1, S. 195–197, 1981.
- [22] R. A. Wagner, M. J. Fischer, *The String-to-String Correction Problem*, J. ACM, Bd. 21, Nr. 1, S. 168–173, 1974.
- [23] V. Vassilevska Williams, *On Some Fine-Grained Questions in Algorithms and Complexity*, in Proc. ICM, Bd. 3, S. 3431–3472, 2018.
- [24] A. Abboud, V. V. Williams, *Popular Conjectures Imply Strong Lower Bounds for Dynamic Problems*, in Proc. IEEE FOCS, S. 434–443, 2014.
- [25] E. Kushilevitz, N. Nisan, *Communication Complexity*, Cambridge University Press, 1997.
- [26] E. M. Clarke, E. A. Emerson, A. P. Sistla, *Automatic Verification of Finite-State Concurrent Systems Using Temporal Logic Specifications*, ACM Trans. Program. Lang. Syst., Bd. 8, Nr. 2, S. 244–263, 1986.
- [27] Synopsys, *Design Compiler User Guide*, Synopsys Inc., Version 2023.03, 2023.
- [28] J. Zhou, G. Cui, S. Hu, Z. Zhang, C. Yang, Z. Liu, L. Wang, C. Li, M. Sun, *Graph Neural Networks: A Review of Methods and Applications*, AI Open, Bd. 1, S. 57–81, 2020.
- [29] M. A. Nielsen, I. L. Chuang, *Quantum Computation and Quantum Information*, Cambridge University Press, 2000.
- [30] T. Lengauer, *Combinatorial Algorithms for Integrated Circuit Layout*, Wiley, 1990.

- [31] C. Y. Lee, *An Algorithm for Path Connections and Its Applications*, IRE Trans. Electron. Comput., Bd. EC-10, Nr. 3, S. 346–365, 1961.
- [32] R. M. Karp, *Reducibility Among Combinatorial Problems*, in Complexity of Computer Computations, Plenum Press, S. 85–104, 1972.
- [33] E. W. Dijkstra, *A Note on Two Problems in Connexion with Graphs*, Numerische Mathematik, Bd. 1, Nr. 1, S. 269–271, 1959.
- [34] R. E. Tarjan, *Depth-First Search and Linear Graph Algorithms*, SIAM J. Comput., Bd. 1, Nr. 2, S. 146–160, 1972.
- [35] C. E. Leiserson, J. B. Saxe, *Retiming Synchronous Circuitry*, Algorithmica, Bd. 6, Nr. 1–6, S. 5–35, 1991.
- [36] G. De Micheli, *Synthesis and Optimization of Digital Circuits*, McGraw-Hill, 1994.
- [37] G. D. Hachtel, F. Somenzi, *Logic Synthesis and Verification Algorithms*, Kluwer Academic Publishers, 1996.
- [38] J. P. Roth, *Diagnosis of Automata Failures: A Calculus and a Method*, IBM J. Res. Dev., Bd. 10, Nr. 4, S. 278–291, 1966.
- [39] M. L. Bushnell, V. D. Agrawal, *Essentials of Electronic Testing for Digital, Memory and Mixed-Signal VLSI Circuits*, Kluwer Academic Publishers, 2000.
- [40] R. E. Bryant, *Graph-Based Algorithms for Boolean Function Manipulation*, IEEE Trans. Comput., Bd. C-35, Nr. 8, S. 677–691, 1986.
- [41] A. Biere, M. Heule, H. van Maaren, T. Walsh (Hrsg.), *Handbook of Satisfiability*, IOS Press, 2009.
- [42] E. F. Moore, *Gedanken-Experiments on Sequential Machines*, in Automata Studies, Princeton University Press, S. 129–153, 1956.
- [43] G. H. Mealy, *A Method for Synthesizing Sequential Circuits*, Bell Syst. Tech. J., Bd. 34, Nr. 5, S. 1045–1079, 1955.
- [44] M. Böhm, E. Speckenmeyer, *A Fast Parallel SAT-Solver – Efficient Workload Balancing I*, Ann. Math. Artif. Intell., Bd. 24, Nr. 1–4, S. 231–248, 1998.
- [45] C. E. Shannon, *A Symbolic Analysis of Relay and Switching Circuits*, Trans. Am. Inst. Electr. Eng., Bd. 57, Nr. 12, S. 713–723, 1938.
- [46] A. V. Aho, J. E. Hopcroft, J. D. Ullman, *The Design and Analysis of Computer Algorithms*, Addison-Wesley, 1974.
- [47] G. S. Tseitin, *On the Complexity of Derivation in Propositional Calculus*, in Studies in Constructive Mathematics and Mathematical Logic, Consultants Bureau, S. 115–125, 1968.
- [48] G.-J. Nam, J. Cong (Hrsg.), *Modern Circuit Placement: Best Practices and Results*, Springer, 2007.

- [49] N. H. E. Weste, D. M. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4. Aufl., Addison-Wesley, 2011.
- [50] C. H. Papadimitriou, *Computational Complexity*, Addison-Wesley, 1994.
- [51] O. Goldreich, *Computational Complexity: A Conceptual Perspective*, Cambridge University Press, 2008.
- [52] D. E. Knuth, *The Art of Computer Programming, Vol. 4A: Combinatorial Algorithms, Part 1*, Addison-Wesley, 2011.